

Chapter 6

Mail

6.1 Introduction 6-1

6.2 Demonstration 6-1

6.2.1 Composing and Sending a Message 6-2

6.2.2 Reading Mail 6-3

6.2.3 Deleting a Message 6-4

6.2.4 Leaving Mail 6-4

6.3 Basic Concepts 6-4

6.3.1 Mailboxes 6-5

6.3.2 Messages 6-5

6.3.3 Modes 6-6

6.3.4 Message-Lists 6-6

6.3.5 Headers 6-7

6.3.6 Command Syntax 6-8

6.4 Using Mail 6-8

6.4.1 Entering and Exiting Mail 6-9

6.4.2 Sending Mail 6-9

6.4.3 Reading Mail 6-10

6.4.4 Disposing of Mail 6-10

6.4.5 Composing Mail 6-10

6.4.6 Forwarding Mail 6-11

6.4.7 Replying to Mail 6-11

6.4.8 Creating Mailing Lists 6-11

6.4.9 Sending Network Mail 6-12

6.4.10 Setting Options 6-12

6.5 Commands 6-12

6.5.1 Getting Help: help and ? 6-12

6.5.2 Reading Mail: p, +, -, and restart 6-12

6.5.3 Finding Out the Number of the Current Message: = 6-14

- 6.5.4 Displaying the First Five Lines: t 6-14
- 6.5.5 Displaying Headers: h 6-14
- 6.5.6 Deleting Messages: d and dp 6-15
- 6.5.7 Undeleting Messages: u 6-15
- 6.5.8 Leaving Mail: q and x 6-15
- 6.5.9 Saving Your Mail: s 6-16
- 6.5.10 Saving Your Mail: w 6-16
- 6.5.11 Saving Your Mail: mb 6-16
- 6.5.12 Saving Your Mail: ho 6-16
- 6.5.13 Printing Your Mail on the Lineprinter: l 6-17
- 6.5.14 Sending Mail: m 6-17
- 6.5.15 Replying to Mail: r and R 6-17
- 6.5.16 Forwarding Mail: f and F 6-17
- 6.5.17 Creating Mailing Lists: a 6-18
- 6.5.18 Setting and Unsetting Options: se and uns 6-18
- 6.5.19 Editing a Message: e and v 6-18
- 6.5.20 Executing Shell Commands: sh and ! 6-19
- 6.5.21 Finding Out the Number of Characters in a Message: si 6-19
- 6.5.22 Changing the Working Directory: cd 6-19
- 6.5.23 Reading Commands from a File: so 6-19

Leaving Compose Mode Temporarily 6-20

- 6.6.1 Getting Help: ~ ? 6-20
- 6.6.2 Printing the Message: ~ p 6-20
- 6.6.3 Editing the Message: ~ e and ~ v 6-20
- 6.6.4 Editing Headers: ~ t, ~ s, ~ c, ~ b, ~ R and ~ h 6-20
- 6.6.5 Adding a File to the Message: ~ r and ~ d 6-22
- 6.6.6 Enclosing Another Message: ~ m and ~ M 6-22
- 6.6.7 Saving the Message in a File: ~ w 6-22
- 6.6.8 Leaving Mail Temporarily: ~ ! and ~ | 6-23
- 6.6.9 Escaping to Mail Command Mode: ~ : 6-23
- 6.6.10 Placing a Tilde at the Beginning of a Line: ~ ^ 6-23

Setting Up Your Environment: The .mailrc File 6-24

- 6.7.1 The Subject Prompt: asksubject 6-24
- 6.7.2 The CC Prompt: askcc 6-24
- 6.7.3 Printing the Next Message: autoprint 6-24
- 6.7.4 Listing Messages in Chronological Order: chron and mchron 6-25

- 6.7.5 Using the Period to Send a Message: dot 6-25
- 6.7.6 Including Yourself in a Group: metoo 6-25
- 6.7.7 Saving Aborted Messages: save 6-25
- 6.7.8 Printing the Version Header: quiet 6-25
- 6.7.9 Choosing an Editor: The EDITOR String 6-25
- 6.7.10 Choosing an Editor: The VISUAL String 6-26
- 6.7.11 Choosing a Shell: The SHELL String 6-26
- 6.7.12 Changing the Escape Character: The Escape String 6-26
- 6.7.13 Setting Page Size: The Page String 6-26
- 6.7.14 Saving Outgoing Mail: The Record String 6-26
- 6.7.15 Keeping Mail in the System Mailbox: autombox 6-26
- 6.7.16 Changing the Top Value: The Toplines String 6-27
- 6.7.17 Sending Mail Over Telephone Lines: ignore 6-27
- 6.8 Using Advanced Features 6-27
 - 6.8.1 Command Line Options 6-27
 - 6.8.2 Using Mail as a Reminder Service 6-28
 - 6.8.3 Handling Large Amounts of Mail 6-28
 - 6.8.4 Maintenance and Administration 6-29
- 6.9 Quick Reference 6-29
 - 6.9.1 Command Summary 6-30
 - 6.9.2 Compose Escape Summary 6-34
 - 6.9.3 Option Summary 6-37



6.1 Introduction

The XENIX **mail** system is a versatile communication facility that allows XENIX users to compose, send, receive, forward, and reply to mail. Users can also create distribution groups and send copies of messages to multiple users. These functions are integrated into XENIX so that all users can quickly and easily communicate with each other.

This chapter is organized to satisfy the needs of both the beginning and advanced user. The first sections discuss basic concepts, tasks, and commands. Later sections discuss advanced topics and provide quick reference to the **mail** program's many functions. The major sections in this chapter are

Demonstration	Shows new users how to get started.
Basic Concepts	Discusses the fundamental ideas and terminology used in mail .
Using Mail	Shows how to perform common mailing procedures such as composing, sending, forwarding, and replying to mail.
Commands	Discusses each mail command.
Leaving Compose Mode Temporarily	Discusses and gives examples of each command available when composing a message. These commands are called <i>compose escapes</i> .
Setting Up Your Environment	Discusses the user's mail startup file and options that customize functions.
Using Advanced Features	Discusses advanced features such as using mail as a reminder service and handling a large volume of mail.
Quick Reference	Summarizes all commands, compose escapes, and options.

6.2 Demonstration

The **mail** command lets you perform two distinct functions: sending mail and deleting mail. In this demonstration, you'll learn to send mail to yourself, read a message, delete it, and exit the **mail** program.

6.2.1 Composing and Sending a Message

To begin, type

```
mail self
```

where *self* is your user name. The **mail** program prompts you for a "Subject:". Type

Sample subject

and press RETURN. Next, type the following lines, ending each with a RETURN:

```
This is a message sent to myself.  
I compose a message by entering lines of text.  
The message is ended by typing CTRL-D on a new line.
```

As you enter the message you can use compose escapes to perform special functions. To get a list of the available compose escapes, type

```
~ ?
```

on a new line. To specify a list of people to receive carbon copies use the `~c` escape. For example, type

```
~c abel
```

To view the message as it will appear when you send it, type

```
~p
```

This will print the following:

```
-----  
Message contains:  
To: self  
Subject: Sample subject  
Cc: abel
```

```
This is a message sent to myself.  
I compose a message by entering lines of text.  
The message is ended by typing CTRL-D on a new line.
```

```
(continue)
```

Finally, to end the message and send it to those you have mentioned in the *To:* and the *Cc:* fields, press CTRL-D by itself on a line. You will exit the **mail** pro-

gram and return to the XENIX shell. Once you have sent mail, there is no way to retrieve it. (Note that if you do not have a user named “abel” on your system, you will receive an error message and XENIX saves the letter in “/usr/self/dead.letter”.)

6.2.2 Reading Mail

Within a short time, when you press RETURN, you should receive the message

```
You have new mail.
```

This indicates that a message has arrived in your system mailbox. To read this message and any others that may have been sent to you, type

```
mail
```

Mail then displays a sign-on message and a list of message headers that look something like this:

```
mail version 3.0 August 30, 1984.  Type ? for help.
1 message:
  1 self    Fri Aug 31 12:26  7/188 "Sample subject"
-

```

The message header indicates

- who sent the message
- when it was sent
- the number of lines and characters
- the subject of the message

The *most recent* message appears at the top of the list. Messages are numbered in ascending order from least to most recent, so the most recent message has the highest number.

To see the message that you sent to yourself press RETURN. (You may have already been sent mail when you first logged on to the system. In this case, you’ll have other messages listed in your mailbox and you must press RETURN again to see each message.)

The **mail** program prints the following:

```
From self Fri Aug 20 12:26:52 1982
To: self
Subject: Sample subject
```

```
This is a message sent to myself.  
I compose a message by entering lines of text.  
The message is ended by typing CTRL-D on a new line.
```

Note that the message you sent to yourself now contains information about the sender of the message—a line telling who sent the message and when it was sent. The next line tells who the message was sent to. A subject and carbon copy (Cc:) field can be specified by the sender. If present, they too are displayed when you read the message.

6.2.3 Deleting a Message

To delete the message, type

```
d
```

6.2.4 Leaving Mail

To exit mail, type

```
q
```

Mail then displays the message

```
0 messages held in /usr/spool/mail/self
```

and returns you to the XENIX shell.

This ends the demonstration. For more detailed information, see the discussions in following sections.

6.3 Basic Concepts

It is much easier to use **mail** if you understand its basic concepts. This section presents the following concepts:

- Mailboxes
- Messages
- Modes
- Command syntax

6.3.1 Mailboxes

It is useful to think of the **mail** system as a typical postal system. What is normally called a post office is called the *system mailbox*. The system mailbox contains a file for each user in the directory */usr/spool/mail*. Your own personal or *user mailbox* is the *mbox* file in your home directory. Mail sent to you is put in your system mailbox; you may choose to save mail in your user mailbox after you have read it. Note that the user mailbox differs from a real mailbox in several respects:

1. *You* decide whether mail is to be placed in the user mailbox; it is not automatically placed there.
2. The user mailbox is *not* the place where mail is initially routed—that place is the system mailbox in the directory */usr/spool/mail*.
3. Mail is not picked up *from* your user mailbox.

6.3.2 Messages

In **mail**, the message is the basic unit of exchange between users. Messages consist of two parts: a heading and a body. The heading contains the following fields:

To:	This field is mandatory and contains one or more user names corresponding to real users to whom you send mail.
Subject:	This optional field contains text describing the message.
Cc:	The carbon copy field contains one or more names of those who will receive copies of a message. Message recipients see these names in the received message. This field can be empty.
Bcc:	The blind carbon copy field contains one or more names of people who will receive copies of a message. Recipients do <i>not</i> see these names in the received messages. This field can be empty.
Return-receipt-to:	The return receipt to: field contains the name or names of those who are to receive an automatic acknowledgement of the message. This field can be empty.

The body of a message is text exclusive of the heading. The body can be empty.

6.3.3 Modes

This section discusses **mail**'s modes of operation.

When you invoke **mail** you are using the shell. You can enter a message from your shell by typing

```
mail john
```

followed by the text of your message. Press RETURN to start a new line, and CTRL-D to send the message.

Such messages are created in **mail**'s *compose mode*. When entering text in compose mode, there are several special keys associated with line editing functions: these are the same special characters that are available to you when executing normal XENIX commands. For example, you can kill the line you are editing by typing the kill character, normally a CTRL-U. To backspace, press either CTRL-H or the BACKSPACE key.

From compose mode, you can issue commands called compose escapes. These are also called *tilde escapes* because the command letters are preceded by a tilde (~). When you execute these commands you are temporarily leaving or escaping from compose mode; hence the name. Note that once you've pressed RETURN to end a line, you cannot change that line from within compose mode; to change it, you must enter edit mode.

The most common way of using **mail** is to just type

```
mail
```

This automatically places you in **mail** *command mode*. In this mode, you are prompted by an underscore for commands that permit you to manipulate your mail.

You can enter *edit mode* from either compose mode or command mode. In edit mode, you edit the body of a message using the full capabilities of an editor. To enter edit mode from command mode, use either the **e** or **edit** command to enter **ed**, or the **v** or **visual** command to enter **vi**. To enter edit mode from compose mode, use the compose escapes **~e** and **~v**, respectively.

6.3.4 Message-Lists

Many **mail** commands take a list of messages as an argument. A *message list* is a list of message numbers, ranges, and names, separated by SPACES or TABs. Message numbers may be either decimal numbers, which directly specify messages, or one of the special characters **^**, **.**, or **\$**, which specify the first, current, or last *undeleted* message, respectively.

A range of messages is two message numbers separated by a dash. To print the first four messages on the screen, type

p 1-4

To print all the messages from the current message to the last message, type

p.-\$

A *name* is a user name. Messages can be printed by specifying the name of the sender. For example, to print each message sent to you by “john”, type

p john

As a shorthand notation, you can specify asterisk (*) to get all *undeleted* messages. For example, to print all messages except those that have been deleted, type

p *

To delete all messages, type

d *

To undelete all messages, type

u *

6.3.5 Headers

When you enter **mail**, a list of message *headers* appears. A header is a single line of text containing descriptive information about a message. (Note that we use the word *heading* to describe the first part of a message, and *header* to describe **mail**'s one-line description of a message.) The information includes

- The number of the message
- The sender
- The date sent
- The number of characters and lines
- The subject (if the message contains a *Subject:* field)

The **headers** command lets you display up to 18 message headers. If there are *more* than 18 messages, the list is divided into an appropriate number of windows. To move forward one window at a time, type

headers +

To move backward one window, type

headers -

6.3.6 Command Syntax

Each **mail** command has its own syntax. Some take no arguments, some take only one, and others take several arguments. The more flexible commands, such as **print**, accept combinations of message lists and user names. For these commands, **mail** first gathers all message numbers and ranges, then finds all messages from any specified user names. The full message list is the intersection of these two sets of messages. Thus, the message list "4-15 miller" matches all messages between 4 and 15 that are from miller.

Each **mail** command is typed on a line by itself, and any arguments follow the command word. The command need not be typed in its entirety—the first command that matches the typed prefix is used. For example, you can type "p" instead of "print" for the **print** command and "h" instead of "headers" for the **headers** command.

After the command itself is typed, one or more spaces should be entered to separate the command from its arguments. If a **mail** command does not take arguments, the arguments you give are ignored. For commands that take message lists as arguments, if no message list is given, the last message printed is used. If it does not satisfy the requirements of the command, the search proceeds forward. If there are no messages forward of the current message, the search proceeds backwards, and if there are no good messages at all, **mail** displays

No applicable messages

6.4 Using Mail

This section describes how to perform some basic tasks when using **mail**. More detailed discussions of each of these commands are presented in later sections.

6.4.1 Entering and Exiting Mail

To begin a **mail** session, type

```
mail
```

The headers for each received message are then displayed one screenful at a time.

To end the **mail** session, use the **quit(q)** command. All messages remain in the system mailbox unless they have been deleted with the **delete(d)** command.

6.4.2 Sending Mail

To send a message, invoke **mail** with the names of the people and groups you want to receive the message. Next, type in your message. When you are finished, press CTRL-D at the beginning of a line. The message is automatically sent to the specified people. While entering the text of your message, you can escape to an editor or perform other useful functions with compose escapes. Section 6.4.5 describes some features of **mail** that help you compose messages.

You can use redirection to mail a file that already exists. Use a command line of the form

```
mail sam bob john <filename
```

where *filename* is the name of the file you are sending.

Note

Be very careful when mailing a file with the input redirection symbol (<). If you accidentally type the output redirection symbol (>), you will overwrite the file, destroying its contents.

If **mail** cannot be delivered to a specified address, you will either be notified immediately, in which case a copy of the undeliverable message is appended to the file *dead.letter*, or you will be notified by return mail, in which case a copy is included in the return mail message.

6.4.3 Reading Mail

To read messages sent to you, type

mail

Mail checks your mail out of the system mailbox and prints out a one-line header of each message, one screenful at a time. The most recent message header appears at the top of the list. You can move forward one message by pressing RETURN or typing "+". To move forward *n* messages use "+*n*". You can move backwards one message with the "-" command or move backwards *n* messages and print with "-*n*". You can also move to any arbitrary message and print it by typing its number.

If new messages arrive while you are in **mail**, the following message appears:

New mail has arrived -- type 'restart' to read.

Type

restart

and the headers of the new messages appear at the top of the list.

6.4.4 Disposing of Mail

After examining a message you can delete it with the **delete** command, reply to it with the **reply(r)** command, forward it with the **forward(f)** command, or skip to the next message by pressing RETURN.

6.4.5 Composing Mail

To compose mail, you must enter compose mode. Do this from XENIX command level by typing

mail *username*

where *username* is the name of a user to whom you want to send mail. From **mail** command mode, you can enter compose mode with the **mail**, **reply**, or **Reply** commands. Once in compose mode, the text that you type is appended one line at a time to the body of the message you are sending. Normal line editing functions are available, including CTRL-U to kill a line and BACK-SPACE to back up one character. To abort a message you have begun, press INTERRUPT twice.

While you are composing a message, **mail** treats lines beginning with the tilde character (~) in a special way. This character introduces commands called compose escapes. Section 6.6 discusses the many uses of compose escapes.

6.4.6 Forwarding Mail

To forward a message, use the **forward(f)** command. For example, to forward a copy of mail from your box to John's, type

```
f john
```

The copy is shifted right one tab stop, and the new message is forwarded to John. John will receive a message heading indicating that you have forwarded the message. The **Forward(F)** command works just like its lowercase counterpart, except that the forwarded message is not shifted right.

6.4.7 Replying to Mail

You can use the **reply** command to set up a response to a message, automatically addressing a reply to the person who sent the original message. You then type in text and send the message by pressing CTRL-D on a line by itself. The **Reply** command works just like its lowercase counterpart, except that the message is sent to others named in the original message's *To:* and *Cc:* fields.

6.4.8 Creating Mailing Lists

You can create personal mailing lists so that, for example, you can send mail to *cohorts* and have it go to a group of people. Such lists are defined by placing an **alias** line like

```
alias cohorts bill bob barry
```

in the file *.mailrc* in your home directory. To display the current list of such aliases use the **alias(a) mail** command. Personal aliases are expanded when the mail is sent. For example, the *To:* field in a message sent to *cohorts* will read

```
To: bill bob barry
```

and not

```
To: cohorts
```

Normally, system-wide aliases are available to all users. These are installed by your system administrator. For more information, see Section 6.8.4.

6.4.9 Sending Network Mail

You can send mail between XENIX machines connected with Micnet or, to sites within the UUCP network. For more information about sending network mail, see the *XENIX Operations Guide*. For more information about UUCP, see the *XENIX Operations Guide*.

6.4.10 Setting Options

Mail has several options that you can set from **mail** command mode or in the file *.mailrc* in your home directory. These options are discussed in Section 6.7.

6.5 Commands

This section describes each of the commands available to you in **mail** command mode. The examples in this section assume you have invoked **mail** and that you have several messages you want to dispose of. Note that in general, **mail** commands can be invoked with either the name of the command or a one- or two-character mnemonic abbreviation.

6.5.1 Getting Help: help and ?

The **help(?)** command prints out a brief summary of all **mail** commands, so if you ever get stuck when you are in **mail** command mode, type

?

or

help

6.5.2 Reading Mail: p, +, -, and restart

To look at a specific message, use the **print(p)** command. For example, pretend you have a header list that looks like this:

3	john	Wed Sep 21 09:21	26/782	"Notice"
2	sam	Tue Sep 20 22:55	6/83	"Meeting"
1	tom	Mon Sep 19 01:23	6/84	"Invite"

Reading from the left, each header contains the message number, who sent it, the day, date, and time it was sent, the number of lines and characters in the message, and its subject.

To examine the second message, type

p 2

This might cause **mail** to respond with

```
Message 2:
From sam Tue Sep 20 22:55 1983
Subject: Meeting
```

Meeting everyone, please don't forget!

To look at message 3, type

+

or to look at message 1, type

-

The commands + and - execute relative to the last message referred to, which in our example was 2. For large numbers of messages, you can skip forward and backward by the number of messages specified as an argument to + and -. For example, typing

+3

skips forward three messages. If you type

p *

then all messages are displayed, since the star (*) matches all messages.

Pressing RETURN prints out the next message in the header list. You can always go to a message and print it by giving its message number or one of the special characters, caret (^), dot (.), or dollar sign (\$). In the example where message 2 is the current message, to print the current message, type

.

To print the first message, type

^

To print the last message, type

\$

When new mail arrives while you are in **mail**, the message "New mail has arrived—type 'restart' to read" appears. If you wish to read the new mes-

sages, type

restart

The headers of the new messages appear at the top of the list.

6.5.3 Finding Out the Number of the Current Message: =

The **number(=)** command prints out the message number of the current message. It takes no arguments.

6.5.4 Displaying the First Five Lines : t

The **top(t)** command prints the first five lines of each addressed message. For example

top 2-12

prints out the first five lines of each of the messages 2 through 12. Note that the number of lines printed out by **top** can be set with the **toplines** option (see Section 6.9.3).

6.5.5 Displaying Headers: h

The **headers(h)** command displays up to 18 headers.

To examine the next set of 18 headers, type

h +

To examine the previous set, type

h -

Both plus and minus take an optional numeric argument that indicates the number of header windows to move forward or backward before printing. If a message list is given, then the **headers** command prints out the header line for each message in the list. For example

h joe

displays all the message headers from joe. The following are some characteristics of the header list:

- Deleted messages do not appear in the listing.
- Messages saved with the **save** command are flagged with an asterisk (*).

- Messages to be saved in your user mailbox are flagged with an “M”.
- If the **autombox** option is set, messages held with the **hold** command are flagged with an “H”.

6.5.6 Deleting Messages: **d** and **dp**

Unless you indicate otherwise, each message you receive is automatically saved in the system mailbox when you quit **mail**. Often, however, you don't want to save messages you have received. To delete messages, use the **delete(d)** command. For example

```
d 1
```

deletes message 1 from the system mailbox.

The **dp** command deletes the current message and prints the next message. It is useful for quickly reading and disposing of mail. Using **dp** is the same as using the **d** command with the **autoprint** option set. See also the **undelete** command.

6.5.7 Undeleting Messages: **u**

The **undelete(u)** command retrieves a message previously deleted with the **d** or **dp** commands. For example, to undelete message 3, type

```
u3
```

You cannot undelete messages from previous **mail** sessions; they have been permanently deleted.

6.5.8 Leaving Mail : **q** and **x**

To leave **mail**, use the **quit(q)** command. All messages are held in your system mailbox, except the following:

- Deleted messages, which are discarded irretrievably.
- Messages marked with the **mbox** command, which are saved in *mbox* in your home directory.
- Messages saved with the **save** and **write** commands, which are deleted from the system mailbox.

If you wish to leave **mail** quickly without altering either your system or user mailbox, use the **exit(x)** command. This returns you to the shell without changing anything: no messages are deleted or saved.

6.5.9 Saving Your Mail: s

The **save(s)** command lets you save messages to a specified file. The **save** command writes out each message to the file given as the last argument on the command line. For example, the following command appends messages 1-5 to the file *letters*:

```
s 1-5 letters
```

The file *letters* is created if it does not already exist. In a header list, each saved message is marked with an asterisk (*).

Save writes the entire message, including the *To:*, *Subject:*, and *Cc:* fields. In comparison, the **write** command (see Section 6.5.10), writes only the bodies of the specified messages.

6.5.10 Saving Your Mail: w

The **write(w)** command writes the body of each message to the file given as the last argument on the command line. Each written message is marked with an asterisk (*). The syntax is similar to that of the **save** command. For example,

```
w 3-17 john elliot book
```

writes out the bodies of all messages from john and elliot in the number range 3-17. They are concatenated to the end of the file named *book*.

6.5.11 Saving Your Mail: mb

The **mbox(mb)** command marks each message specified in a message list, so that all are saved in the user mailbox when a **quit** command is executed. Message headers are marked with an "M" to show that they are to be saved in *mbox*.

6.5.12 Saving Your Mail: ho

The **hold(ho)** command takes a message list and marks each message so that it is saved in your system mailbox instead of deleted or saved in *mbox* when you quit. Since files in the system mailbox are saved by default, use **hold only** when you have also set the **autombox** option.

6.5.13 Printing Your Mail on the Lineprinter: l

The **lpr(l)** command paginates and prints out messages to the lineprinter. It takes a message list as its argument, then paginates and prints out each message. For example

```
l doug
```

prints out each message from the user doug on the lineprinter.

6.5.14 Sending Mail: m

To send mail to a user, use the **mail(m)** command. This sends mail in the manner described for the **reply** command, except that you supply a list of recipients either as an argument or by entering them in the *To:* field. All compose escapes work in **mail**. Note that the **mail** command is in most ways identical to typing **mail users** at the XENIX command level.

6.5.15 Replying to Mail: r and R

Often, you want to respond to a message right away. The **reply(r)** command is useful for this purpose: it takes a message list and sends mail to the author of each message. The original message's subject field is copied as the reply's subject. Each message is composed in compose mode; thus all compose escapes work in **reply**, and messages are terminated by pressing CTRL-D.

The **Reply(R)** command works just like its lowercase counterpart, except that copies of the reply are also sent to everyone shown in the original message's *To:* and *Cc:* fields.

6.5.16 Forwarding Mail: f and F

To forward a copy of a message, use the **forward(f)** command. This sends a copy of the current message to the specified users. For example, to forward the current message to someone whose login name is john, type

```
f john
```

John will receive the forwarded message, along with a heading showing that you are the one who forwarded it. Inside the new message, the forwarded message is indented one tab stop. An optional message number can also be given. For example

```
f 2 john bill
```

forwards message 2 to john and bill.

The **Forward(F)** command is identical to the lowercase **forward** command,

except that the forwarded message is not indented.

6.5.17 Creating Mailing Lists: **a**

The **alias(a)** command links a group of names with the single name given by the first argument, thus creating a mailing list. For example, you could type

```
alias beatles john paul george ringo
```

so that whenever you used the name *beatles* in a destination address (as in “**mail** beatles”), it would expand to list the four names associated with the “beatles” alias. With no arguments, **alias** prints out all currently-defined aliases. With one argument, it prints out the users defined by the given alias.

You will probably want to define aliases in the startup file, *.mailrc*, so that you don't have to redefine them each time you invoke **mail**. See Section 6.7 for more information.

6.5.18 Setting and Unsetting Options: **se** and **uns**

Mail switch and string options can be set with the **mail** commands **set** and **unset**. A switch option is either on or off (set or unset). String options are strings of characters that are assigned values with the syntax “option=string”. Multiple options may be specified on a line. Setting or unsetting options this way are only valid for that particular run of **mail**. You may find it more useful to place **set** and **unset** commands in the file *.mailrc* in your home directory, where they become your own personal default options each time you invoke **mail**. For example, you might have a **set** command that looked like this

```
set dot metoo topline=10 SHELL=bin/sh
```

The options *dot* and *metoo* are switch options; *toplines* and *SHELL* are string options.

The command

```
set ?
```

prints out a list of the available options. See Section 6.7 for descriptions of these options.

6.5.19 Editing a Message: **e** and **v**

To edit individual messages using the text editor, use the **edit(e)** command. It takes a message list and processes each message in turn by writing it to a temporary file. The editor, **ed**, is then automatically invoked so that you can edit the temporary file. When you finish editing the message, write the message out, then quit the editor. **Mail** reads the message back into the message buffer

and removes the temporary file.

It is often useful to be able to invoke either a line or visual editor, depending on the type of terminal you are using. To invoke **vi**, you can use the **visual(v)** command. The operation of the **visual** command is otherwise identical to that of the **edit** command.

6.5.20 Executing Shell Commands: **sh** and **!**

To execute a shell command without leaving **mail**, precede the command with an exclamation point. For example

```
!date
```

prints the current date without leaving **mail**. To enter a new shell, type

```
sh
```

To exit from this new shell and return to **mail** command mode, press CTRL-D.

6.5.21 Finding Out the Number of Characters in a Message: **si**

The **size(si)** command prints out the number of characters in each message in a message list. For example, the command “**si 1-4**” might print out

```
4: 234
3: 1000
2: 23
1: 456
```

6.5.22 Changing the Working Directory: **cd**

The **cd** command changes the working directory to the name of the directory you give it as an argument. If no argument is given, the directory is changed to your home directory. This command works just like the normal XENIX **cd** command. (Note that exiting **mail** returns you to the directory from which you entered **mail**; thus the **mail cd** command works only within **mail**.) You may want to place a **cd** command in your **.mailrc** file so that you always begin executing **mail** from within the same directory.

6.5.23 Reading Commands from a File: **so**

The **source(so)** command reads in **mail** commands from a named file. Normally, these commands are **alias**, **set**, and **unset** commands.

6.6 Leaving Compose Mode Temporarily

While composing a message to be sent to others, it is often useful to print a message, invoke the text editor on a partial message, execute a shell command, or perform some other function. **Mail** provides these capabilities through compose escapes (sometimes called tilde escapes) which consist of a tilde (`~`) at the beginning of a line, followed by a single character that specifies the function to be performed. These escapes are available *only* when you are composing a new message. They have no meaning when you are in **mail** command mode. The available compose escapes are as follows:

6.6.1 Getting Help: `~?`

The help escape

`~?`

prints a brief summary of the available compose escapes on your screen.

6.6.2 Printing the Message: `~p`

The print escape

`~p`

prints a line of dashes and the heading and body of the message you are composing.

6.6.3 Editing the Message: `~e` and `~v`

The editor escapes, `~e` and `~v`, copy the current message into a temporary file so it can be edited. After modifying the message to your satisfaction, write it out and quit the editor. **Mail** responds with the message

(continue)

after which you may continue composing your message.

6.6.4 Editing Headers: `~t`, `~s`, `~c`, `~b`, `~R` and `~h`

To add additional names to the list of message recipients, type the escape

`~t name1 name2 ...`

You can name as many additional recipients as you wish. Note that users originally on the recipient list will still receive the message: you cannot remove anyone from the recipient list with `~t`. To remove a recipient, use the `~h` command, which is discussed later in this section.

You can replace or add a subject field by using the `~s` escape

`~s line-of-text`

This replaces any previous subject with *line-of-text*. The subject, if given, appears near the top of the message, prefixed with the heading "Subject:". You can see what the message looks like by using `~p`, which prints out all heading fields along with the body of the text.

You may occasionally prefer to list certain people as recipients of carbon copies of a message rather than direct recipients. The escape

`~c name1 name2 ...`

adds the named people to the Cc: list. The escape

`~cc name1 name2 ...`

performs an identical function. Similarly, the escape

`~b name1 name2 ...`

adds the named people to the Bcc: (Blind carbon copy) list. The people on this list receive a copy of the message, but are not mentioned anywhere in the message you send. Remember that you can always execute a `~p` escape to see what the message looks like.

The escape

`~R`

adds or changes the person or persons named in the return-receipt-to: field.

The recipients of the message are given in the To: field; the subject is given in the Subject: field, carbon copy recipients are given in the Cc: field and the return receipt recipient in the Return-receipt-to: field. If you wish to edit these in ways impossible with the `~t`, `~s`, `~c`, and `~R` escapes, you can use

`~h`

where *h* stands for "heading." The escape `~h` prints To: followed by the current list of recipients and leaves the cursor at the end of the line. If you type in ordinary characters, they are appended to the end of the current list of recipients. You can also use the normal XENIX command line editing characters to edit these fields; you can erase existing heading text by backspacing over it.

When you press RETURN, **mail** advances to the Subject: field, where the same rules apply. Another RETURN brings you to the Cc: field, another brings you

to the *Bcc:* field, and yet another to the Return-receipt-to: field. Each of these fields can be edited in the same way. Finally, another RETURN leaves you appending text to the end of your message body. As always, you can use `~p` to print the current text of the heading fields along with the body of the message.

6.6.5 Adding a File to the Message: `~r` and `~d`

It is often useful to be able to include the contents of another file in your message. The escape

`~r filename`

is provided for this purpose, and causes the named file to be appended to your current message. **Mail** displays a message if the file doesn't exist or can't be read. If the read is successful, **mail** prints the number of lines and characters appended to your message.

As a special case of `~r`, the escape

`~d`

reads in the file *dead.letter* in your home directory. This is often useful because **mail** copies the text of your message buffer to *dead.letter* whenever you abort the creation of a message by either typing two consecutive interrupts or entering a `~q` escape.

6.6.6 Enclosing Another Message: `~m` and `~M`

If you are sending mail from within **mail**'s command mode, you can insert a message sent to you into the message you are currently composing. For example, you might type

`~m 4`

This reads message 4 into the message you are composing, shifted right one tab stop. The escape

`~M 4`

performs the same function, but with no right shift. You can name any non-deleted message or list of messages.

6.6.7 Saving the Message in a File: `~w`

To save the current text of a message body in a file, use

`~w filename`

Mail writes out the message body to the specified file, then prints the number of lines and characters written to the file. The `~w` escape does *not* write the message heading to the file.

6.6.8 Leaving Mail Temporarily: `~!` and `~|`

To temporarily escape to the shell, use the escape

`~!command`

This executes *command* and returns you to **mail** compose mode without altering your message. If you wish to filter the body of your message through a shell command, use

`~|command`

This pipes your message through the command and uses the output as the new text of your message. If the command produces no output, **mail** assumes that something is wrong, retains the old version of your message, and prints

(continue)

6.6.9 Escaping to Mail Command Mode: `~:`

To temporarily escape to **mail** command mode, use either of the following escapes:

`~:mail-command`

`~_mail-command`

You can then execute any **mail** command that you want. Note that this escape will not work in most cases if you enter compose mode from the XENIX shell. It depends on the command used (**set** and **unset** will work), but most commands that involve message lists are not allowed. You will receive the message

May not execute *cmd* while composing

6.6.10 Placing a Tilde at the Beginning of a Line: `^^`

If you wish to send a message that contains a line beginning with a tilde, you must type it twice. For example, typing

`^^` This line begins with a tilde.

appends

~ This line begins with a tilde.

to your message. The escape character can be changed to a different character with the **escape** option. (For information on how to set options, see Section 6.7.) If the escape character is not a tilde, then this discussion applies to that character and not the tilde.

6.7 Setting Up Your Environment: The .mailrc File

Whenever **mail** is invoked, it first reads the file */usr/lib/mail/mailrc* then the file *.mailrc* in the user's home directory. System-wide aliases are defined in */usr/lib/mail/mailrc*. Personal aliases and **set** options are defined in *.mailrc*. The following is a sample *.mailrc* file:

```
alias office bill steve karen
alias cohorts john mary bob beth mike
set dot askcc
cd
```

6.7.1 The Subject Prompt: asksubject

The **asksubject** switch causes prompting for the subject of each message before you enter compose mode. If you respond to the prompt with a RETURN, no subject field is sent.

6.7.2 The CC Prompt: askcc

The **askcc** switch causes prompting for additional carbon copy recipients when you finish composing a message. Responding with a RETURN signals your satisfaction with the current list. Pressing INTERRUPT prints

```
interrupt
(continue)
```

so that you can return to editing your message.

6.7.3 Printing the Next Message: autoprint

The **autoprint** switch causes the **delete** command to behave like **dp**. After deleting a message, the next message in the list is automatically printed. Printing also occurs automatically after execution of an **undelete** command.

6.7.4 Listing Messages in Chronological Order: **chron** and **mchron**

The **chron** switch causes messages to be listed in chronological order. By default, messages are listed with the most recent first. Set **chron** when you want messages to appear in the order they were received.

The **mchron** switch, like **chron**, prints messages in chronological order, but lists them in the opposite order, that is, highest-numbered, or most recent, first. This is useful if you keep a large number of messages in your mailbox and you wish to list the headers of the most recently received mail first but read the messages themselves in chronological order.

6.7.5 Using the Period to Send a Message: **dot**

The **dot** switch lets you use a period (.) as an end-of-transmission character, as well as CTRL-D. This option is helpful for users accustomed to this convention from editing with the editor, **ed**.

6.7.6 Including Yourself in a Group: **metoo**

Usually, when an alias expands, the sender's name is removed from the expansion. Setting the **metoo** option causes the sender to be included in the group.

6.7.7 Saving Aborted Messages: **save**

The **nosave** switch prevents aborted messages from being appended to the file *dead.letter* in your home directory; messages are saved by default. Messages are aborted from compose mode by two interrupts or a **~q** compose escape.

6.7.8 Printing the Version Header: **quiet**

The **quiet** switch suppresses the printing of the version header when **mail** is first invoked.

6.7.9 Choosing an Editor: The **EDITOR** String

The *EDITOR* string contains the pathname of the text editor to use in the **edit** command and **~e** escape. If not defined, then the default editor is used. For example

```
set EDITOR=/bin/ed
```

6.7.10 Choosing an Editor: The VISUAL String

The *VISUAL* string contains the pathname of the text editor used in the *visual* command and *~v* escape. For example

```
set VISUAL=/bin/vi
```

By default, *vi* is the editor used.

6.7.11 Choosing a Shell: The SHELL String

The *SHELL* string contains the name of the shell to use in the *!* command and the *~!* escape. A default shell is used if this option is not defined. For example

```
set SHELL=/bin/sh
```

6.7.12 Changing the Escape Character: The Escape String

The *escape* string defines the character to use in place of the tilde (*~*) to denote compose escapes. For example

```
set escape=*
```

With this setting, the asterisk becomes the new compose escape character.

6.7.13 Setting Page Size: The Page String

The *page* string causes messages to be displayed in pages of size *n* lines. You are prompted with a question mark between pages. Pressing RETURN causes the next page of the current message to be printed. By default this paging feature is turned off.

6.7.14 Saving Outgoing Mail: The Record String

The *record* string sets the pathname of the file used to record all outgoing mail. If undefined, outgoing mail is not copied and saved. For example

```
set record=/usr/john/recordfile
```

With this setting, all outgoing mail is automatically appended to the file */usr/john/recordfile*.

6.7.15 Keeping Mail in the System Mailbox: autombox

The *autombox* switch determines whether messages remain in the system mailbox when you exit *mail*. If you set *autombox*, examined messages are automatically placed in the *mbox* file in your home directory (your user mail-

box) and removed from the system mailbox when you quit.

6.7.16 Changing the Top Value: The Toplines String

The **toplines** string sets the number of lines of a message to be printed out with the **top** command. By default, this value is five. For example

```
set topline=10
```

With this setting, ten lines of each message are printed out when the **top** command is used.

6.7.17 Sending Mail Over Telephone Lines: ignore

The **ignore** switch causes interrupt signals from your terminal to be ignored and echoed as at-signs (@). This switch is normally used only when communicating with **mail** over telephone lines.

6.8 Using Advanced Features

This section discusses advanced features of **mail** useful to those familiar with the XENIX **mail** system.

6.8.1 Command Line Options

One very useful command line option to **mail** is the **subject switch**(-s) With this switch you can specify a subject on the command line. For example, you could send a file named *letter* with the subject line, "Important Meeting at 12:00", by typing the following:

```
mail -s "Important Meeting at 12:00" john bob mike <letter
```

To include other header fields in your message, you can use the following options:

- b user** Adds the blind carbon copy field to the message header.
- c user** Adds the carbon copy field to the message header.
- r user** Adds the return-receipt to: field to the message header.

None of the above options may be specified more than once on a mail command line. If multiple arguments are required for an option, the entire argument set must be enclosed in quotes, as in

```
mail -r "self" -b user3 -c "user1 user4" user user2
```

Mail also allows you to edit files of messages by using the **-f** switch on the command line. For example

mail -f filename

causes **mail** to edit *filename* and the command

mail -f

causes **mail** to read *mbox* in your home directory. All **mail** commands except **hold** are available to edit the messages. When you type the **quit** command, **mail** writes the updated file back.

If you send mail over a noisy phone line, you may notice that bad characters are transmitted. Many of these will be the characters that abort messages: the RUBOUT or DEL character. To ignore these bad characters, invoke **mail** with the **-i** switch.

When you enter the **mail** program (as opposed to sending a message from command level), two command line options are available:

- R** Makes the mail session read-only, preventing alteration of the mail being read.
- u user** Reads in *user's* mail instead of your own.

6.8.2 Using Mail as a Reminder Service

You can also use **mail** as a reminder service. XENIX automatically examines the file named *calendar* in each user's home directory and looks for lines containing either today or tomorrow's date. These lines are sent by **mail** as reminders of important events.

You can also write shell procedures to have **mail** signal the completion of a job. For information about writing shell procedures, see Chapter 7, "The Shell."

6.8.3 Handling Large Amounts of Mail

You must periodically examine your user mailbox contents to decide whether messages are still relevant; you should only save *important* mail in your user mailbox.

One organizational method is to save mail in files organized by sender, by topic, or by a combination of the two. Create these files in a separate **mail** directory; you can access these mailbox files with the **mail -f filename** switch. However, be forewarned—this approach to organizing mail quickly eats up disk space.

6.8.4 Maintenance and Administration

The following programs and files make up the XENIX mail system:

<code>/usr/bin/mail</code>	Mail program
<code>/usr/lib/mail/mailrc</code>	Mail system initialization file
<code>/usr/spool/mail/*</code>	System mailbox files
<code>/usr/name/dead.letter</code>	File where undeliverable mail is deposited
<code>/usr/name/mbox</code>	User mailbox
<code>/usr/name/mailrc</code>	User mail initialization file
<code>/usr/lib/mail/mailhelp.cmd</code>	Mail command help file
<code>/usr/lib/mail/mailhelp.esc</code>	Mail compose escape help file
<code>/usr/lib/mail/mailhelp.set</code>	Mail option help file
<code>/usr/lib/mail/aliases</code>	System-wide aliases
<code>/usr/lib/mail/aliases.hash</code>	System-wide alias database
<code>/usr/lib/mail/faliases</code>	Forwarding aliases
<code>/usr/lib/mail/maliases</code>	Machine aliases
<code>/usr/lib/mail/maliases.hash</code>	Optional machine aliases database

A system-wide distribution list is kept in `/usr/lib/mail/aliases`. A system administrator is usually in charge of this list. These aliases are kept in a vastly different syntax from `.mailrc`, and are expanded when mail is sent. You will normally need special permission to change system-wide aliases.

6.9 Quick Reference

The following sections provide quick reference to the available commands, compose escapes, and options.

6.9.1 Command Summary

The following are the names and syntax for each command, the abbreviated form (in brackets), and a short description. Many commands have optional arguments; most can be executed without any arguments at all. In particular, commands that take a message-list argument will default to the current message if no message list is given. In the following descriptions, boldface denotes the name of a command, compose escape or option. Italics represent arguments to commands or compose escapes. The vertical bar separates arguments from which you may select. All other text should be read literally.

Command	Function
RETURN	Prints the next message.
<i>n</i>	Goes to the <i>n</i> th message, and prints it.
+ <i>n</i>	[+] With no <i>n</i> argument, goes to the next message and prints it. If given a numeric argument <i>n</i> , goes to the <i>n</i> th next message and prints it.
- <i>n</i>	[-] With no <i>n</i> argument, goes to the previous message and prints it. If given a numeric argument <i>n</i> , goes to the <i>n</i> th previous message and prints it.
^	Prints the first message.
\$	Prints the last message.
=	Prints the message number of the current message.
?	Prints the summary of mail commands in <i>/usr/lib/mail/mailhelp.cmd</i> .
! <i>shell-cmd</i>	Executes the shell command that follows. No space is needed after the exclamation point.
Alias <i>users</i>	Prints system-wide aliases for users. At least one user must be specified. You can substitute "A" for "Alias."

alias *name users*

[a] Aliases *users* to *name*. With no arguments, prints all currently defined aliases. With one argument, prints the users aliased by the given name argument.

cd *directory*

[c] Changes the user's working directory to the specified directory. If no directory is given, then changes to the user's home directory.

delete [*msg-list*]

[d] Deletes each message in the given message list.

dp *msg-list*

Deletes the current message and prints the next message.

echo *path*

Expands shell metacharacters.

edit [*msg-list*]

[e] Takes the given message list and points the text editor at each message in turn. On return to command mode, the edited message is read back in. See also the **visual** command.

exit[!]

[x] Immediately returns to the shell without modifying the system mailbox, the user mailbox, or a file specified with the **-f** switch.

file

[fi] Prints the name of the mailbox file.

forward *msg-num user-list*

[f] Takes a *user-list* argument and forwards the current message to each name. The message sent to each is indented and shows that the sender has passed it on. The *msg-num* argument is optional, and is used to forward the numbered message instead of the default message.

Forward *msg-num user-list*

[F] Same as **forward** except that the message is not indented.

headers [<i>+n</i> <i>-n</i> <i>msg-list</i>]	[h] With no argument, lists the current range of headers, which is an 18-message group. If a plus (+) argument is given, then the next 18-message group is printed, and if a minus (−) argument is given, the previous 18-message group is printed. Both plus and minus accept an optional numeric argument indicating the number of header windows to move forward or backward. If a message list is given, then the message header for each message in the list is printed.
help	Same as ?. Prints the summary of mail commands in <i>/usr/lib/mail/mailhelp.cmd</i> .
hold [<i>msg-list</i>]	[ho] Takes a message list and marks each message to be saved in the user's system mailbox instead of in <i>mbox</i> .
list	Prints a list of mail commands.
lpr [<i>msg-list</i>]	[l] Prints each of the messages in the required message list on the lineprinter. Messages are piped through <i>pr</i> before being printed.
mail [<i>user-list</i>]	[m] Takes an optional user-list argument and sends mail to each name after entering compose mode.
mbox [<i>msg-list</i>]	[mb] Marks messages given in the message-list argument to be saved in the user mailbox when a quit is executed. Message headers contain an initial letter "M" to show that they are to be saved.
move <i>msg-list msg-num</i>	Places the messages specified in <i>msg-list</i> after the message specified in <i>msg-num</i> . If <i>msg-num</i> is 0, <i>msg-list</i> moves to the top of the mailbox.
print [<i>msg-list</i>]	[p] Takes a message list and prints each message on the user's terminal.

 quit

[q] Terminates the **mail** session, retaining all nondeleted, unsaved messages in the system mailbox. If the **autombox** option is set, then examined messages are saved in the user mailbox, deleted messages are discarded, and all messages marked with the **hold** command are retained in the system mailbox.

If you are executing a **quit** while editing a mailbox file with the **-f** flag, the mailbox file is rewritten and the user returns to the shell.

reply *[msg-list]*

[r] Takes a message list and sends mail to each message author just like the **mail** command.

Reply *[msg-list]*

[R] Sends a reply to users named in the To: and Cc: fields, as well as the original sender.

 restart

Reads in mail that arrives during the current mail session.

save *[msg-list] filename*

[s] Takes an optional message list and a filename and appends each message in turn to the end of the file. The default message is the current message.

set

[se] Prints a list of available options.

set *option-list*

[se] With no arguments, displays options currently set. Otherwise, sets option. Arguments are of the form *option=value* if the option is a string option, or just *option* if the option is a switch. Multiple options may be set on one line. To see available options, type “set ?.”

shell

[sh] Invokes an interactive version of the shell.

 size *msg-list*

[si] Takes a message list and prints the size in characters of each message.

source <i>file</i>	[so] Reads and executes mail commands from the named file.
string <i>string</i> [<i>msg-list</i>]	Searches for <i>string</i> in <i>msg-list</i> . If no <i>msg-list</i> is specified, all undeleted messages are searched. Ignores case in search.
top	[t] Takes a message list and prints the top five lines. The number of lines printed is set by the variable <i>toplines</i> .
undelete [<i>msg-list</i>]	[u] Takes a message list and marks each one as not being deleted. Each message in the list must previously have been deleted.
unset <i>options</i>	[uns] Takes a list of option names and discards their remembered values; this is the opposite of set .
visual [<i>msg-list</i>]	[v] Invokes the vi editor on each message specified in the list.
whois	Looks up a list of target mail recipients and prints the real names or descriptions of each recipient. If the first character of the first argument is alphabetic, the arguments are looked up without change. Otherwise, the arguments are assumed to be a message list, in the format specified in the <i>Mail User's Guide</i> . For each message in the list, the "From" person is extracted from the header and added to the list of users to be searched.
write <i>msg-list filename</i>	[w] Writes the message bodies of messages given by the message list to the file given by <i>filename</i> .

6.9.2 Compose Escape Summary

Compose escapes are used when composing messages to perform special functions. They are only recognized at the beginning of lines. The escape character can be set with the **escape** string option. (See Section 6.7.12.) Abbreviations for each escape are in brackets.

Here is a summary of the compose escapes:

<code>~string</code>	Inserts the string of text in the message prefaced by a single tilde (<code>~</code>).
<code>~?</code>	Prints out help for compose escapes on terminal.
<code>~.</code>	Same as CTRL-D on a new line.
<code>~!command</code>	Executes a shell command, then returns to compose mode.
<code>~ command</code>	Pipes the message body through the command as a filter. Replaces the message body with the output of the filter. If the command gives no output or terminates abnormally, retains the original message body.
<code>~_mail-command</code>	Executes a mail command, then returns to compose mode.
<code>~:mail-command</code>	Executes a mail command, then returns to compose mode.
<code>~alias</code>	[<code>~a</code>] Prints a list of private aliases.
<code>~alias aliasname</code>	[<code>~a</code>] Prints the names included in private <i>aliasname</i> .
<code>~alias aliasname users</code>	[<code>~a</code>] Adds <i>users</i> to private <i>aliasname</i> list.
<code>~Alias</code>	[<code>~A</code>] Performs aliasing by first examining private aliases and then system-wide aliases using all three global alias files. Only the final result is printed (non-local mail recipients will have the complete delivery path printed). The user list is taken from header fields.
<code>~Alias users</code>	[<code>~A</code>] Performs aliasing by first examining private aliases and then system-wide aliases using all three global alias files. Only the final result is printed (non-local mail recipients will have the complete delivery path printed). At least one user must be specified.
<code>~bcc name ...</code>	[<code>~b</code>] Adds the given names to the Bcc: field.
<code>~cc name ...</code>	[<code>~c</code>] Adds the given name to the Cc: field.

~ dead	[~d] Reads the file <i>dead.letter</i> from your home directory into the message.
~ editor	[~e] Invokes the line editor on the message being sent. Exiting the editor returns the user to compose mode.
~ headers	[~h] Allows the user to modify each field in the message heading.
~ message <i>msg-list</i>	[~m] Reads the named messages into the message being sent, shifted right one tab. If no messages are specified, reads the current message.
~ Message <i>msg-list</i>	[~M] Same as ~ message except with no right shift.
~ print	[~p] Prints the message buffer prefaced by the message heading.
~ Print	[~P] Prints the real names or descriptions (in parentheses) after each recipient.
~ quit	[~q] Aborts the message being sent, copying the message to <i>dead.letter</i> in your home directory if the <i>nosave</i> option is not set.
~ read <i>filename</i>	[~r] Reads the named file into the message
~ Return <i>name</i>	[~R] Adds the given names to the Return-receipt-to: field.
~ shell	[~sh] Invokes a shell.
~ subject <i>string</i>	[~s] Causes the named string to become the current subject field.
~ to <i>name ...</i>	[~t] Adds the given names to the To: field.
~ visual	[~v] Invokes the <i>vi</i> editor to edit the message buffer. Exiting the editor returns the user to compose mode.
~ write <i>filename</i>	[~w] Writes the message body to the named file.

6.9.3 Option Summary

Options are controlled with the **set** and **unset** commands. An option is either a switch or a string. A switch is either on or off, while a string option has a value that is a pathname, a number, or a single character.

askcc	Causes prompting for additional carbon copy recipients at the end of each message. Pressing RETURN retains the current list.
asksubject	Causes prompting for the subject of each message you send. The subject is a line of text terminated by a RETURN.
autombox	Usually messages are retained in the system mailbox when the user quits. However, if this option is set , examined messages are automatically appended to the user mailbox.
autoprint	Causes the delete command to behave like dp . Thus, after deleting (or undeleting) a message, the next one is printed automatically.
chron	Causes messages to be listed in chronological order.
dot	Causes a single period on a new line to act as the EOT character. The normal end-of-transmission character, CTRL-D, still works.
EDITOR=	Pathname of the text editor to use in the edit command and ~e escape. If not defined, then a default editor is used.
escape=char	If defined, sets <i>char</i> as the character to use in place of the tilde (~) to denote compose escapes.
ignore	Causes interrupt signals from your terminal to be ignored and echoed as at-signs (@).
mchron	Causes messages to be listed in numerical order (most recently received first), but displayed in chronological order.
metoo	Normally, before sending, the name of the sender is removed from alias expansions. If <i>metoo</i> is set, then the name of the sender is <i>not</i> removed.

nosave	Prevents saving of the message buffer in the file <i>dead.letter</i> in the home directory, after two consecutive interrupts or a ~q escape.
page=<i>n</i>	Specifies the number of lines (<i>n</i>) to be printed in a "page" of text when displaying messages.
quiet	Suppresses the printing of the version when mail is first invoked.
record=	Sets the pathname of the file used to record all outgoing mail. If not defined, then outgoing mail is <i>not</i> copied.
SHELL=	Pathname of the shell to use in the ! command and the ~! escape. A default shell is used if this option is not defined.
toplines=	Sets the number of lines of a message to be printed with the top command. Default is five lines.
verify	Causes each target mail recipient to be verified. This option permits errors made while composing messages to be corrected or ignored.
VISUAL=	Pathname of the text editor to use in the visual command and ~v escape. The default is for the vi editor.

Chapter 7

The Shell

- 7.1 Introduction 7-1
- 7.2 Basic Concepts 7-1
 - 7.2.1 How Shells Are Created 7-1
 - 7.2.2 Commands 7-2
 - 7.2.3 How the Shell Finds Commands 7-2
 - 7.2.4 Using Metacharacters on Command Lines 7-3
 - 7.2.5 Quoting Mechanisms 7-4
- 7.3 Redirecting Input and Output 7-5
 - 7.3.1 Standard Input and Output 7-6
 - 7.3.2 Diagnostic and Other Outputs 7-6
 - 7.3.3 Command Lines and Pipelines 7-7
 - 7.3.4 Command Substitution 7-9
- 7.4 Shell Variables 7-10
 - 7.4.1 Positional Parameters 7-10
 - 7.4.2 User-Defined Variables 7-10
 - 7.4.3 Predefined Special Variables 7-13
- 7.5 The Shell State 7-14
 - 7.5.1 Changing Directories 7-15
 - 7.5.2 The .profile File 7-15
 - 7.5.3 Execution Flags 7-15
- 7.6 A Command's Environment 7-16
- 7.7 Invoking the Shell 7-17
- 7.8 Passing Arguments to Shell Procedures 7-17
- 7.9 Controlling the Flow of Control 7-19

- 7.9.1 Using the if Statement 7-21
- 7.9.2 Using the case Statement 7-22
- 7.9.3 Conditional Looping: while and until 7-23
- 7.9.4 Looping Over a List: for 7-23
- 7.9.5 Loop Control: break and continue 7-25
- 7.9.6 End-of-File and exit 7-25
- 7.9.7 Command Grouping: Parentheses and Braces 7-26
- 7.9.8 Input/Output Redirection and Control Commands 7-27
- 7.9.9 Transfer to Another File and Back: The Dot (.) Command 7-27
- 7.9.10 Interrupt Handling: trap 7-27
- 7.10 Special Shell Commands 7-30
- 7.11 Creation and Organization of Shell Procedures 7-32
- 7.12 More About Execution Flags 7-33
- 7.13 Supporting Commands and Features 7-34
 - 7.13.1 Conditional Evaluation: test 7-34
 - 7.13.2 Echoing Arguments 7-36
 - 7.13.3 Expression Evaluation: expr 7-36
 - 7.13.4 True and False 7-37
 - 7.13.5 In-Line Input Documents 7-37
 - 7.13.6 Input / Output Redirection Using File Descriptors 7-38
 - 7.13.7 Conditional Substitution 7-38
 - 7.13.8 Invocation Flags 7-40
- 7.14 Effective and Efficient Shell Programming 7-40
 - 7.14.1 Number of Processes Generated 7-41
 - 7.14.2 Number of Data Bytes Accessed 7-42
 - 7.14.3 Shortening Directory Searches 7-42
 - 7.14.4 Directory Search Order and the PATH Variable 7-42
 - 7.14.5 Good Ways to Set Up Directories 7-43
- 7.15 Shell Procedure Examples 7-44
- 7.16 Shell Grammar 7-52

7.1 Introduction

When users log into XENIX, they communicate with the shell command interpreter, **sh**. This interpreter is a XENIX program that supports a very powerful command language. Each invocation of this interpreter is called a shell; and each shell has one function: to read and execute commands from its standard input.

Because the shell gives the user a high-level language in which to communicate with the operating system, XENIX can perform tasks unheard of in less sophisticated operating systems. Commands that would normally have to be written in a traditional programming language can be written with just a few lines in a shell procedure. In other operating systems, commands are executed in strict sequence. With XENIX and the shell, commands can be

- Combined to form new commands
- Passed positional parameters
- Added or renamed by the user
- Executed within loops or executed conditionally
- Created for local execution without fear of name conflict with other user commands
- Executed in the background without interrupting a session at a terminal

Furthermore, commands can redirect command input from one source to another and redirect command output to a file, terminal, printer, or other command. This provides flexibility in tailoring a task for a particular purpose.

7.2 Basic Concepts

The shell itself (i.e., the program that reads your commands when you log in or that is invoked with the **sh** command) is a program written in the C language; it is not part of the operating system proper, but an ordinary user program.

7.2.1 How Shells Are Created

In XENIX, a process is an executing entity complete with instructions, data, input, and output. All processes have lives of their own, and may even start (or *fork*) new processes. Thus, at any given moment several processes may be executing, some of which are “children” of other processes.

Users log into the operating system and are assigned a *shell* from which they execute. This shell is a personal copy of the shell command interpreter that is

reading commands from the keyboard: in this context, the shell is simply another process.

In the XENIX multitasking environment, files may be created in one phase and then sent off to be processed in the background. This allows the user to continue working while programs are running.

7.2.2 Commands

The most common way of using the shell is by typing simple commands at your keyboard. A *simple command* is any sequence of arguments separated by spaces or tabs. The first argument specifies the name of the command to be executed. Any remaining arguments, with a few exceptions, are considered as arguments to that command. For example, the following command line requests printing of the files *allan*, *barry*, and *calvin*:

```
lpr allan barry calvin
```

If the first argument of a command names a file that is a compiled, executable program, the shell, as parent, creates a child process that immediately executes that program. If the file is executable, but not compiled, it is assumed to be a shell procedure, i.e., a file of ordinary text containing shell command lines. In this case, the shell spawns another instance of itself (a *subshell*) to read the file and execute the commands inside it.

From the user's viewpoint, compiled programs and shell procedures are invoked in exactly the same way. The shell determines which implementation has been used, rather than requiring the user to do so.

7.2.3 How the Shell Finds Commands

The shell normally searches for commands in three distinct locations in the file system:

- in the current directory
- in the directory */bin*
- in the */usr/bin* directory

For example, the **pr** and **man** commands are actually the files */bin/pr* and */usr/bin/man*, respectively. A more complex pathname may be given, either to locate a file relative to the user's current directory, or to access a command with an absolute pathname. If the command name begins with a slash (/) (e.g., */bin/sort* or */cmd*), the prepending is not performed. Instead, a single attempt is made to execute the command as named.

This mechanism gives the user a convenient way to execute public commands and commands in or near the current directory, as well as the ability to execute any accessible command, regardless of its location in the file structure. Because the current directory is usually searched first, anyone can possess a private version of a public command without affecting other users. Similarly, the creation of a new public command does not affect a user who already has a private command with the same name. To change the sequence of directories searched, reset the shell PATH variable. (Shell variables are discussed in Section 7.4).

7.2.4 Using Metacharacters on Command Lines

Command arguments are very often filenames. Sometimes, a group of related files have similar, but not identical, filenames. The XENIX Shell lets you use a special set of characters, called *metacharacters*, to easily specify a group of similarly named files as a command argument.

You can substitute a metacharacter for a portion of a command line, and XENIX will find all files that match the search specification. XENIX uses the following metacharacters:

Metacharacter	Matches
*	any string (including the null string)
?	any one character
[...]	any character enclosed in the brackets
[x-y]	any character in the range enclosed in the brackets

For example

*	Matches all names in the current directory
temp	Matches all names containing "temp"
[a-f]*	Matches all names beginning with "a" through "f"
*.c	Matches all names ending in ".c"
/usr/bin/?	Matches all single-character names in /usr/bin

Pattern matching has some restrictions. If the first character of a filename is a period (.), it can be matched only by an argument that literally begins with a

period. If a pattern does not match any filenames, then the pattern itself is printed out as the result of the match.

Note that directory names should not contain the following characters:

`* ? [ ]`

If these characters are used, then infinite recursion may occur during pattern matching attempts.

7.2.5 Quoting Mechanisms

The characters `<, >, *, ?, [` and `]` have special meanings to the shell. To remove the special meaning of these characters requires some form of quoting. This is done by using single, close quotation marks (`'`) or double quotation marks (`"`) to surround a string. (Single, open quotation marks (`'`) are used only for command substitution in the shell and do not hide the special meanings of any characters.)

All characters within single quotation marks are interpreted literally. Thus

```
echostuff='echo $? $*; ls * | wc'
```

assigns the string

```
echo $? $*; ls * | wc
```

to the variable *echostuff*, but does *not* result in any other commands being executed.

Within double quotation marks, certain characters retain their special meaning, while all other characters are interpreted literally. The characters that retain their special meaning are the dollar sign (`$`), the backslash (`\`), the single, close quotation mark (`'`), and the double quotation mark (`"`) itself. Thus, within double quotation marks, variables are expanded and command substitution takes place (both topics are discussed in later sections). However, any commands in a command substitution are unaffected by double quotation marks, so that characters such as star (`*`) retain their special meaning.

To hide the special meaning of the dollar sign (`$`) and single and double quotation marks within double quotation marks, precede these characters with a backslash (`\`). Outside of double quotation marks, preceding a character with a backslash is equivalent to placing single quotation marks around that character. A backslash (`\`) followed by a newline causes that newline to be ignored and is equivalent to a space. The backslash-newline pair is therefore useful in allowing continuation of long command lines.

The following list shows how the shell interprets quoted material:

Input	Shell interprets as
'	The single, open quotation mark (`)
"	The double quotation mark (")
`echo one`	the one word " `echo one` "
"\""	The double quotation mark (")
"`echo one`"	the one word "one"
"`"	illegal (expects another `)
one two	the two words "one" & "two"
"cne two"	the one word "one two"
'one two'	the one word "one two"
'one * two'	the one word "one * two"
"cne * two"	the one word "one * two"
`echo one`	the one word "one"

7.3 Redirecting Input and Output

In general, most commands do not know or care whether their input or output is coming from or going to a terminal or a file. Thus, a command can be used conveniently either at a terminal or in a pipeline. A few commands vary their actions depending on the nature of their input or output, either for efficiency, or to avoid useless actions (such as attempting random access I/O on a terminal or a pipe).

When a command begins execution, it assumes that three files are already open. Each file is associated with a number, called a *file descriptor*:

File	File Descriptor
Standard Input	0
Standard Output	1
Diagnostic Output (or Standard Error)	2

A child process normally inherits these files from its parent. All three files are initially connected to the terminal (0 to the keyboard, 1 and 2 to the terminal screen). The shell permits the files to be redirected elsewhere before control is passed to an invoked command.

7.3.1 Standard Input and Output

An argument to the shell of the form “<*file*” or “>*file*” opens the specified file as the standard input or output (in the case of output, destroying the previous contents of *file*, if any). An argument of the form “>>*file*” directs the standard output to the end of *file*, thus providing a way to append data to the file without destroying its existing contents. In either of the two output cases, the shell creates *file* if it does not already exist. Thus

```
>output
```

alone on a line creates a zero-length file. The following appends to file *log* the list of users who are currently logged on:

```
who >> log
```

Such redirection arguments are only subject to variable and command substitution; neither blank interpretation nor pattern matching of filenames occurs after these substitutions. This means that

```
echo `this is a test` > *.gal
```

produces a one-line file named **.gal*. Similarly, an error message is produced by the following command, unless you have a file with the name “?”:

```
cat < ?
```

So remember, special characters are *not* expanded in redirection arguments because the arguments are scanned by the shell *before* pattern recognition and expansion takes place. For more information on redirection, refer to the *XENIX C Library Guide*.

7.3.2 Diagnostic and Other Outputs

Diagnostic output from XENIX commands is normally directed to the file associated with file descriptor 2. (There is often a need for an error output file that is different from standard output so that error messages do not get lost down pipelines.) You can redirect this error output to a file by immediately prepending the number of the file descriptor (2 in this case) to either output redirection symbol (> or >>). The following line appends error messages from the *cc* command to the file named *ERRORS*:

```
cc testfile.c 2>>ERRORS
```

There can be no spaces or tabs between the file descriptor number and the redirection symbol, or the number will be passed as a command argument.

You can redirect output associated with any of the first ten file descriptors (numbered 0-9). For instance, if **cmd** puts output on file descriptor 9, then the following line will direct that output to the file *savedata*:

```
cmd 9>savedata
```

A command often generates standard output and error output, and might even have some other output, perhaps a data file. In this case, one can **redirect** independently all the different outputs. Suppose, for example, that **cmd** directs its standard output to file descriptor 1, its error output to file descriptor 2, and builds a data file on file descriptor 9. The following would direct each of these three outputs to a different file:

```
cmd >standard 2>error 9>data
```

7.3.3 Command Lines and Pipelines

A sequence of commands separated by the vertical bar (|) makes up a *pipeline*. In a pipeline consisting of more than one command, each command is run as a separate process connected to its neighbors by *pipes*, that is, the output of each command (except the last one) becomes the input of the next command in line.

A *filter* is a command that reads its standard input, transforms it in some way, then writes it as its standard output. A pipeline normally consists of a series of filters. Although the processes in a pipeline are permitted to execute in parallel, each program needs to read the output of its predecessor. Many commands operate on individual lines of text, reading a line, processing it, writing it out, and looping back for more input. Some must read large amounts of data before producing output; **sort** is an example of the extreme case that requires all input to be read before any output is produced.

The following is an example of a typical pipeline:

```
nroff -mm text|col|lpr
```

Nroff is a text formatter in the XENIX Text Processing System whose output may contain reverse line motions, **col** converts these motions to a form that can be printed on a terminal lacking reverse-motion capability, and **lpr** does the actual printing. The flag **-mm** indicates one of the commonly used formatting options, and *text* is the name of the file to be formatted.

The following examples illustrate the variety of effects that can be obtained by combining a few commands in the described manner:

- `who`
Prints the list of logged-in users on the terminal screen.
- `who>>log`
Appends the list of logged-in users to the end of file *log*.
- `who|wc -l`
Prints the number of logged-in users.
- `who|pr`
Prints a paginated list of logged-in users.
- `who|sort`
Prints an alphabetized list of logged-in users.
- `who|grep bob`
Prints the list of logged-in users whose login names contain the string "bob".
- `who|grep bob|sort|pr`
Prints an alphabetized, paginated list of logged-in users whose login names contain the string bob.
- `{ date; who|wc -l; }>>log`
Appends (to file *log*) the current date, followed by the count of logged-in users. Be sure to place a space after the left brace and a semicolon before the right brace.
- `who|sed -e 's/ .*//'|sort|uniq -d`
Prints only the login names of all users who are logged in more than once. Note the use of `sed` as a filter to remove characters trailing the login name from each line. (The ".*" in the `sed` command is preceded by a space.)

The **who** command does not *by itself* provide options to yield all these results—they are obtained by combining **who** with other commands. Note that **who** just serves as the data source in these examples. As an exercise, replace "who|" with "</etc/passwd" in the examples to see how a file can be used as a data source in the same way. Notice that redirection arguments may appear anywhere on the command line, even at the start. This means that

`<infile>outfile sort|pr`

is the same as

```
sort|pr <infile >outfile
```

7.3.4 Command Substitution

Any command line can be placed within single, open quotation marks (``...``) so that the output of the command replaces the quoted command line itself. This concept is known as *command substitution*. The command or commands enclosed between single, open quotation marks are first executed by the shell and then their output replaces the whole expression, single, open quotation marks and all. This feature is often used to assign values to shell variables. (Shell variables are described in Section 7.4.) For example

```
today=`date`
```

assigns the string representing the current date to the variable “today”; for example “Tue Nov 27 16:01:09 EST 1982”. The following command saves the number of logged-in users in the shell variable *users*:

```
users=`who | wc -l`
```

Any command that writes to the standard output can be enclosed in single, open quotation marks. Single, open quotation marks may be nested, but the inside sets must be escaped with backslashes (`\`). For example

```
logmsg=`echo Your login directory is `pwd``
```

assigns the string “your login directory is *name of login directory*” to the variable “logmsg.” Shell variables can also be given values indirectly by using the **read** and **line** commands. The **read** command takes a line from the standard input (usually your terminal) and assigns consecutive words on that line to any variables named.

For example

```
read first init last
```

takes an input line of the form

```
G. A. Snyder
```

and has the same effect as typing

```
first=G. init=A. last=Snyder
```

The **read** command assigns any excess “words” to the last variable.

The **line** command reads a line of input from the standard input and then echoes it to the standard output.

7.4 Shell Variables

The shell has several mechanisms for creating variables. A variable is a name representing a string value. Certain variables are referred to as *positional parameters*; these are the variables that are normally set only on the command line. Other shell variables are simply names to which the user or the shell itself may assign string values.

7.4.1 Positional Parameters

When a shell procedure is invoked, the shell implicitly creates *positional parameters*. The name of the shell procedure itself in position zero on the command line is assigned to the positional parameter \$0. The first command argument is called \$1, and so on. You can use the **shift** command to access arguments in positions numbered higher than nine. For example, the following shell script might be used to cycle through command line switches and then process all succeeding files:

```
while test '$1'
do case $1 in
    -a) A=aooption ; shift ;;
    -b) B=booption ; shift ;;
    -c) C=cooption ; shift ;;
    -*) echo "bad option" ; exit 1 ;;
    *) process rest of files
esac
done
```

One can explicitly force values into these positional parameters by using the **set** command. For example,

```
set abc def ghi
```

assigns the string "abc" to the first positional parameter, \$1, the string "def" to \$2, and the string "ghi" to \$3. Note that \$0 may not be assigned a value in this way—it always refers to the name of the shell procedure; or in the login shell, to the name of the shell.

7.4.2 User-Defined Variables

The shell also recognizes alphanumeric variables to which string values may be assigned. A simple assignment has the syntax

name=string

Thereafter, *\$name* will yield the value *string*. A *name* is a sequence of letters, digits, and underscores that begins with a letter or an underscore. No spaces surround the equal sign (=) in an assignment statement. Note that positional parameters may not appear on the left side of an assignment statement; they can be set only as described in the previous section.

More than one assignment may appear in an assignment statement, but beware: *the shell performs the assignments from right to left*. Thus, the following command line results in the variable “A” acquiring the value “abc”:

```
A=$B B=abc
```

The following are examples of simple assignments. Double quotation marks around the right-hand side allow spaces, tabs, semicolons, and newlines to be included in a string, and allows *variable substitution* (also known as *parameter substitution*) to occur. This means that references to positional parameters and other variable names that are prefixed by a dollar sign (\$) are replaced by the corresponding values, if any. Single quotation marks inhibit variable substitution.

```
MAIL=/usr/mail/gas
echovar="echo $1 $2 $3 $4"
stars=*****
asterisks='$stars'
```

In the above example, the variable “echovar” has as its value the string consisting of the values of the first four positional parameters, separated by spaces, plus the string “echo”. No quotation marks are needed around the string of asterisks being assigned to *stars* because pattern matching (expansion of asterisk, the question mark, and brackets) does not apply in this context. Note that the value of *\$asterisks* is the literal string “\$stars”, *not* the string “*****”, because the single quotation marks inhibit substitution.

In assignments, spaces are not reinterpreted after variable substitution. The following example results in *\$first* and *\$second* having the same value:

```
first='a string with embedded spaces'
second=$first
```

In accessing the values of variables, you can enclose the variable name in braces {...} to delimit the variable name from any following string. In particular, if the character immediately following the name is a letter, digit, or underscore, then the braces are required. For example, examine the following input

```
a='This is a string'
echo "${a}ent test of variables."
```

Here, the `echo` command prints:

```
    This is a stringent test of variables.
```

If no braces were used, the shell would substitute a null value for “`$aent`” and print:

```
    test of variables.
```

The following variables are maintained by the shell. Some of them are set by the shell, and all of them can be reset by the user.

HOME	Specifies the name of the user's <i>login directory</i> , that is, the directory that becomes current following login. This variable is initialized by the <code>login</code> program. <code>Cd</code> , without arguments, switches to the <code>\$HOME</code> directory. Using this variable helps keep full pathnames out of shell procedures. This is of great benefit when pathnames are changed, either to balance disk loads or to reflect administrative changes.
IFS	Specifies which characters are <i>internal field separators</i> . These are the characters the shell uses during blank interpretation. (If you want to parse some delimiter-separated data easily, you can set <code>IFS</code> to include that delimiter.) The shell initially sets <code>IFS</code> to include the blank, tab, and newline characters.
MAIL	The pathname of a file where your mail is deposited. If <code>MAIL</code> is set, then the shell checks to see if anything has been added to the file it names and announces the arrival of new mail each time you return to command level (e.g., by leaving the editor). <code>MAIL</code> must be set by the user and <i>exported</i> . (The <code>export</code> command is discussed in in Section 7.6.) (The presence of mail in the standard mail file is also announced at login, regardless of whether <code>MAIL</code> is set.)
PATH	Specifies the search path used by the shell in finding commands. Its value is an ordered list of directory pathnames separated by colons. The shell initializes <code>PATH</code> to the list <code>/bin:/usr/bin</code> where a null argument appears in front of the first colon. A null anywhere in the path list represents the current directory. On some systems, a search of the current directory is <i>not</i> the default and the <code>PATH</code> variable is initialized instead to <code>/bin:/usr/bin</code> . If you wish to search your current directory last rather than first, use


```
PATH=/bin:/usr/bin::
```

Here, the two colons together represent a colon followed by a null, followed by a colon, thus naming the current directory. You could possess a personal directory of commands (say, *\$HOME/bin*) and cause it to be searched *before* the other three directories by using

```
PATH=$HOME/bin::/bin:/usr/bin
```

“PATH” is normally set in your *.profile* file.

- PS1 Specifies the primary *prompt* string. If the shell is interactive, it prompts with the value of PS1 when it expects input. The default value of PS1 is “\$ ” (a dollar sign (\$) followed by a blank).
- PS2 Specifies the secondary prompt string. If the shell expects more input when it encounters a newline in its input, it prompts with the value of PS2. The default value for this variable is “> ” (a greater-than symbol followed by a space).

In general, you should be sure to export all of the above variables so that their values are passed to all shells created from your login. Use **export** at the end of your *.profile* file, for example

```
export HOME IFS MAIL PATH PS1 PS2
```

7.4.3 Predefined Special Variables

Several variables have special meanings; the following are set *only* by the shell:

- \$# Records the number of arguments passed to the shell, not counting the name of the shell procedure itself. For instance, \$# yields the number of the highest set positional parameter. Thus

```
sh cmd a b c
```

automatically sets \$# to 3. One of its primary uses is in checking for the presence of the required number of arguments:

```

if test $# -lt 2
then
    echo 'two or more args required'; exit
fi

```

\$? Contains the exit status (also referred to as “return code,” “exit code,” or “value”) of the last command executed. Its value is a decimal string. Most XENIX commands return zero to indicate successful completion. The shell itself returns the current value of **\$?** as its exit status.

\$\$ The process number of the current process. Because process numbers are unique among all existing processes, this string is often used to generate unique names for temporary files. XENIX provides no mechanism for the automatic creation and deletion of temporary files; a file exists until it is explicitly removed. Temporary files are generally undesirable; the XENIX pipe mechanism is far superior for many applications. However, the need for uniquely-named temporary files does occasionally occur.

The following example illustrates the recommended practice of creating temporary files. Note that the directories */tmp* and */usr/tmp* are cleared out if the system is rebooted.

```

#       use current process id
#       to form unique temp file
temp=/usr/tmp/$$
ls > $temp
#       commands here, some of which use $temp
rm $temp
#       clean up at end

```

! The process number of the last process run in the background (using the ampersand (&)). This is a string containing from one to five digits.

\$- A string consisting of names of execution flags currently turned on in the shell. For example, **\$-** might have the value “xv” if you are tracing your output.

7.5 The Shell State

The state of a shell is determined by the values of positional parameters, user-defined variables, environment variables, modes of execution, and the current working directory. The state may be altered in various ways, such as changing the working directory with the **cd** command, setting several flags, and by reading commands from the special file, *.profile*, in your login directory.

7.5.1 Changing Directories

The `cd` command changes the current directory to the one specified as its argument. You can place `cd` within parentheses to cause a subshell to change to a different directory and execute some commands without affecting the original shell.

For example, the first sequence below copies the file `/etc/passwd` to `/usr/you/passwd`; the second example first changes directory to `/etc` and then copies the file:

```
cp /etc/passwd /usr/you/passwd
(cd /etc ; cp passwd /usr/you/passwd)
```

Note the use of parentheses. Both command lines have the same effect.

7.5.2 The .profile File

The file named `.profile` is read each time you log in to XENIX. It is normally used to execute special one-time-only commands and to set and export variables to all later shells. Only after commands are read and executed from `.profile` does the shell read commands from the standard input (usually the terminal).

7.5.3 Execution Flags

The `set` command lets you alter the behavior of the shell by setting certain shell flags. In particular, the `-x` and `-v` flags may be useful when invoking the shell as a command from the terminal. To set flags `-x` and `-v`, type

```
set -xv
```

To turn off the same flags, type

```
set +xv
```

These two flags have the following functions:

- v** Prints input lines as they are read by the shell. This flag is particularly useful for isolating syntax errors. The commands on each input line are executed after that input line is printed.
- x** Prints commands and their arguments as they execute. (Shell control commands, such as `for`, `while`, etc., are not printed, however.) Note that `-x` causes a trace of only those commands that are actually executed, whereas `-v` prints each line of input until a syntax error is detected.

The **set** command is also used to set these and other flags within shell procedures.

7.6 A Command's Environment

All variables and their associated values that are known to a command at the beginning of its execution make up its *environment*. This environment includes variables that the command inherits from its parent process and variables specified as *keyword parameters* on the command line that invokes the command.

The variables that a shell passes to its child processes are those that have been named as arguments to the **export** command. The **export** command places the named variables in the environments of both the shell *and* all its future child processes.

Keyword parameters are variable-value pairs that appear in the form of assignments, normally *before* the procedure name on a command line. Such variables are placed in the environment of the procedure being invoked. For example

```
#      keycommand
echo $a $b
```

This is a simple procedure that echoes the values of two variables. If it is invoked as

```
a=key1 b=key2 keycommand
```

then the resulting output is

```
key1 key2
```

Keyword parameters are *not* counted as arguments to the procedure and do not affect \$#.

A procedure may access the value of any variable in its environment. However, if changes are made to the value of a variable, these changes are not reflected in the environment; they are local to the procedure in question. In order for these changes to be placed in the environment that the procedure passes to *its* child processes, the variable must be named as an argument to the **export** command within that procedure. To obtain a list of variables that have been made exportable from the current shell, type

```
export
```

You will also receive a list of variables that have been made **readonly**. To get a list of name-value pairs in the current environment, type either

`printenv`

or

`env`

7.7 Invoking the Shell

The shell is a command and may be invoked in the same way as any other command:

`sh proc [arg...]`

A new instance of the shell is explicitly invoked to read *proc*. Arguments, if any, can be manipulated.

`sh -v proc [arg...]`

This is equivalent to putting “set -v” at the beginning of *proc*. It can be used in the same way for the -x, -e, -u, and -n flags.

`proc [arg ...]`

If *proc* is an executable file, and is not a compiled executable program, the effect is similar to that of

`sh proc args`

An advantage of this form is that variables exported in the shell will still be exported from *proc* when this form is used (because the shell only forks to read commands from *proc*). Thus any changes made within *proc* to the values of exported variables will be passed on to subsequent commands invoked from *proc*.

7.8 Passing Arguments to Shell Procedures

When a command line is scanned, any character sequence of the form *\$n* is replaced by the *n*th argument to the shell, counting the name of the shell procedure itself as \$0. This notation permits direct reference to the procedure name and to as many as nine positional parameters. You can process additional arguments with the **shift** command or **for** loop.

The **shift** command shifts arguments to the left; i.e., the value of \$1 is thrown away, \$2 replaces \$1, \$3 replaces \$2, and so on. The highest-numbered positional parameter becomes *unset* (\$0 is never shifted). For example, in the following shell procedure, *ripple*, **echo** writes its arguments to the standard out-

put:

```
#      ripple command
while test $# != 0
do
    echo $1 $2 $3 $4 $5 $6 $7 $8 $9
    shift
done
```

Lines that begin with a number sign (#) are comments. The looping command, **while**, is discussed in Section 7.9.3 of this chapter. If the procedure were invoked with

```
ripple a b c
```

it would print

```
a b c
b c
c
```

The special shell variable, star (\$*), causes substitution of all positional parameters except \$0. Thus, the **echo** line in the *ripple* example could be written more compactly as

```
echo $*
```

These two **echo** commands are *not* equivalent: the first prints at most nine positional parameters; the second prints *all* of the current positional parameters. The shell star variable (\$*) is more concise and less error-prone. One obvious application is in passing an arbitrary number of arguments to a command. For example

```
wc $*
```

counts the words in each file named on the command line.

It is important to understand the sequence of actions used by the shell in scanning command lines and substituting arguments. The shell first reads input up to a newline or semicolon, and then parses that much of the input. Variables are replaced by their values and command substitution (via single, open quotation marks) is attempted. I/O redirection arguments are detected, acted upon, and deleted from the command line. Next, the shell scans the resulting command line for *internal field separators*, that is, for any characters specified by IFS to break the command line into distinct arguments; *explicit* null arguments (specified by "" or '') are retained, while implicit null arguments

resulting from evaluation of variables that are null or not set are removed. Then filename generation occurs with all metacharacters being expanded. The resulting command line is then executed by the shell.

Sometimes command lines are built within a shell procedure. In this case, it is sometimes useful to have the shell rescan the command line after all the initial substitutions and expansions have been performed. The special command **eval** is available for this purpose. **Eval** takes a command line as its argument and simply rescans the line, performing the specified variable or command substitutions. Consider the following (simplified) situation:

```
command=who
output='| wc -l'
eval $command $output
```

This segment of code results in the execution of the command line

```
who|wc -l
```

The output of **eval** cannot be redirected. However, uses of **eval** can be nested so that a command line can be evaluated several times.

7.9 Controlling the Flow of Control

Several shell commands implement a variety of control structures useful in controlling the flow of control in shell procedures. Before describing these structures, a few terms need to be defined.

A *simple command* is any single irreducible command specified by the name of an executable file. I/O redirection arguments can appear in a simple command line and are passed to the shell, *not* to the command.

A *command* is a simple command or any of the shell control commands described in this section.

A *pipeline* is a sequence of one or more commands separated by vertical bars (`|`). In a pipeline, the standard output of each command but the last is connected (by a *pipe*) to the standard input of the next command. Each command in a pipeline is run separately; the shell waits for the last command to finish. The exit status of a pipeline is nonzero if the exit status of either the first or last process in the pipeline is nonzero.

A *command list* is a sequence of one or more pipelines separated by a semicolon (`;`), an ampersand (`&`), an “and-if” symbol (`&&`), or an “or-if” (`||`) symbol, and optionally terminated by a semicolon or an ampersand. A semicolon causes sequential execution of the previous pipeline. This means that the shell waits for the pipeline to finish before reading the next pipeline. On the other hand, the ampersand (`&`) causes asynchronous background execu-

tion of the preceding pipeline. Thus, both sequential and background execution are allowed. A background pipeline continues execution until it terminates voluntarily, or until its processes are killed.

Other uses of the ampersand include off-line printing, background compilation, and generation of jobs to be sent to other computers. For example, if you type

```
nohup cc prog.c&
```

you may continue working while the C compiler runs in the background. Under *csh*, a command line ending with an ampersand is immune to interrupts or quits that you might generate by typing *INTERRUPT* or *QUIT*. It is also immune to logouts with *CTRL-D*. However, *CTRL-D* will abort the command if you are operating over a dial-up line. In this case, it is wise to make the command immune to hang-ups (i.e., logouts) as well. The **nohup** command is used for this purpose. If **nohup** is *not* used and you log out from a dial-up line while **cc** is still executing, **cc** will be killed and your output will disappear.

Use the ampersand operator with restraint, especially on heavily-loaded systems.

The and-if and or-if (**&&** and **||**) operators cause conditional execution of pipelines. Both of these are of equal precedence when evaluating command lines (but both are of lower precedence than the ampersand (**&**) and the vertical bar (**|**)). In the command line

```
cmd1 || cmd2
```

the first command, *cmd1*, is executed and its exit status examined. Only if *cmd1* fails (i.e., has a nonzero exit status) is *cmd2* executed. Thus, this is a more terse notation for

```
if      cmd1
      test $? != 0
then
      cmd2
fi
```

The and-if operator (**&&**) yields a complementary test. For example, in the following command line

```
cmd1 && cmd2
```

the second command is executed only if the first *succeeds* (and has a zero exit status). In the following sequence, each command is executed in order until one fails:

cmd1 && cmd2 && cmd3 && ... && cmdn

A simple command in a pipeline may be replaced by a command list enclosed in either parentheses or braces. The output of all the commands so enclosed is combined into one stream that becomes the input to the next command in the pipeline. The following line formats and prints two separate documents:

```
{ nroff -mm text1; nroff -mm text2; }|lpr
```

Note that a space is needed after the left brace and that a semicolon should appear before the right brace.

7.9.1 Using the if Statement

The shell provides structured conditional capability with the **if** command. The simplest **if** command has the following form:

```
if command-list
then command-list
fi
```

The command list following the **if** is executed and if the last command in the list has a zero exit status, then the command list that follows **then** is executed. The word **fi** indicates the end of the **if** command.

To cause an alternative set of commands to be executed when there is a nonzero exit status, an **else** clause can be given with the following structure

```
if command-list
then command-list
else command-list
fi
```

Multiple tests can be achieved in an **if** command by using the **elif** clause, although the **case** statement (See Section 7.9.2) is better for large numbers of tests. For example

```
if      test -f "$1"
#                               is $1 a file?
then    pr $1
elif    test -d "$1"
#                               else, is $1 a directory?
then    (cd $1; pr *)
else    echo $1 is neither a file nor a directory
fi
```

This example is executed as follows: if the value of the first positional param-

eter is a filename (**-f**), then print that file; if not, then check to see if it is the name of a directory (**-d**). If so, change to that directory (**cd**) and print all the files there (**pr***). Otherwise, **echo** the error message.

The **if** command may be nested (but be sure to end each one with a **fi**). The new lines in the above examples of **if** may be replaced by semicolons.

The exit status of the **if** command is the exit status of the last command executed in any **then** clause or **else** clause. If no such command was executed, **if** returns a zero exit status.

Note that an alternate notation for the **test** command uses brackets to enclose the expression being tested. For example, the previous example might have been written as follows:

```
if      [ -f"$1" ]
#                               is $1 a file?
then    pr $1
elif    [ -d "$1" ]
#                               else, is $1 a directory?
then    (cd $1; pr *)
else    echo $1 is neither a file nor a directory
fi
```

Note that a space after the left bracket and one before the right bracket are essential in this form of the syntax.

7.9.2 Using the case Statement

A multiple test conditional is provided by the **case** command. The basic format of the **case** statement is

```
case string in
    pattern ) command-list ;;
    ...
    pattern ) command-list ;;
esac
```

The shell tries to match *string* against each pattern in turn, using the same pattern-matching conventions as in filename generation. If a match is found, the command list following the matched pattern is executed; the double semicolon (;;) serves as a break out of the **case** and is required after each command list except the last. Note that only one pattern is ever matched, and that matches are attempted in order, so that if a asterisk (*) is the first pattern in a **case**, no other patterns are looked at.

More than one pattern may be associated with a given command list by specifying alternate patterns separated by vertical bars (|):

```

case $i in
    *.c)      cc $i
             ;;
    *.h | *.sh)
             : do nothing
             ;;
    *)        echo "$i of unknown type"
             ;;
esac

```

In the above example, no action is taken for the second set of patterns because the null, colon (:) command is specified. The asterisk (*) is used as a default pattern, because it matches any word.

The exit status of **case** is the exit status of the last command executed in the **case** command. If no commands are executed, then **case** has a zero exit status.

7.9.3 Conditional Looping: while and until

A **while** command has the general form

```

while command-list
do
    command-list
done

```

The commands in the first *command-list* are executed, and if the exit status of the last command in that list is zero, then the commands in the second *command-list* are executed. This sequence is repeated as long as the exit status of the first *command-list* is zero. A loop can be executed as long as the first *command-list* returns a nonzero exit status by replacing **while** with **until**.

Any newline in the above example may be replaced by a semicolon. The exit status of a **while** (or **until**) command is the exit status of the last command executed in the second *command-list*. If no such command is executed, **while** (or **until**) has a zero exit status.

7.9.4 Looping Over a List: for

To perform a set of operations for each file in a set of files, or execute a command once for each of several arguments, use the **for** command. The **for** command has the format

```

for variable in word-list
do
    command-list
done

```

Here *word-list* is a list of strings separated by spaces. The commands in the *command-list* are executed once for each word in the *word-list*. *Variable* takes on as its value each word from the word list, in turn. The word list is fixed after it is evaluated the first time. For example, the following **for** loop causes each of the C source files *xec.c*, *cmd.c*, and *word.c* in the current directory to be compared with a file of the same name in the directory */usr/src/cmd/sh*:

```
for CFILE in xec cmd word
do    diff ${CFILE}.c /usr/src/cmd/sh/${CFILE}.c
done
```

Note that the first occurrence of **CFILE** immediately after the word **for** has no preceding dollar sign, since the name of the variable is wanted and not its value.

You can omit the “**in word-list**” part of a **for** command; this causes the current set of positional parameters to be used in place of *word-list*. This is useful when writing a command that performs the same set of commands for each of an unknown number of arguments. Create a file named *echo2* that contains the following shell script:

```
for word
do echo $word$word
done
```

Give *echo2* execute status:

```
chmod +x echo2
```

Now type the following command:

```
echo2 ma pa bo fi yo no so ta
```

The output from this command is

```
mama
papa
bobo
fifi
yoyo
nono
soso
tata
```

7.9.5 Loop Control: break and continue

You can use the **break** command to terminate execution of a **while** or a **for** loop. You can use **continue** to request the execution of the next iteration of the loop. These commands are effective only when they appear between **do** and **done**.

The **break** command terminates execution of the smallest (i.e., innermost) enclosing loop, causing execution to resume after the nearest following unmatched **done**. Exit from *n* levels is obtained by **break n**.

The **continue** command causes execution to resume at the nearest enclosing **for**, **while**, or **until** statement (i.e., the one that begins the innermost loop containing the **continue**). You can also specify an argument *n* to **continue** and execution will resume at the *n*th enclosing loop:

```
# This procedure is interactive.
# "Break" and "continue" commands are used
# to allow the user to control data entry.
while true #loop forever
do      echo "Please enter data"
        read response
        case "$response" in
            "done") break
                    # no more data
                    ;;
            "")      # just a carriage return,
                    # keep on going
                    continue
                    ;;
            *)        # process the data here
                    ;;
        esac
done
```

7.9.6 End-of-File and exit

When the shell reaches the end-of-file in a shell procedure, it terminates execution, returning to its parent the exit status of the last command executed prior to the end-of-file. The top level shell is terminated by typing a CTRL-D, which is equivalent to logging out.

The **exit** command simply reads to the end-of-file and returns, setting the exit status to the value of its argument, if any. Thus, a procedure can be terminated normally by placing "exit 0" at the end of the file.

7.9.7 Command Grouping: Parentheses and Braces

There are two methods for grouping commands in the shell: parentheses and braces. Parentheses cause the shell to create a subshell that reads the enclosed commands. Both the right and left parentheses are recognized wherever they appear in a command line—they can appear as literal parentheses *only* when enclosed in quotation marks. For example, if you type

```
garble(stuff)
```

the shell prints an error message. Quoted lines, such as

```
garble>("stuff")"
"garble(stuff)"
```

are interpreted correctly. Other quoting mechanisms are discussed in Section 7.2.5.

Command grouping is useful when you want to perform operations but not affect the values of current shell variables. It also lets you temporarily change the working directory and execute commands in the new directory without having to return to the current directory.

The current environment is passed to the subshell and variables that are exported in the current shell are also exported in the subshell. Thus

```
CURRENTDIR=`pwd`; cd /usr/docs/otherdir;
nohup nroff doc.n | lpr& ; cd $CURRENTDIR
```

and

```
(cd /usr/docs/otherdir; nohup nroff doc.n | lpr&)
```

accomplish the same result: a copy of */usr/docs/otherdir/doc.n* is sent to the lineprinter. (*nroff* is a command available in the XENIX Text Processing System.) However, the second example automatically puts you back in your original working directory. In the second example, spaces or newlines surrounding the parentheses are allowed but not necessary. When entering a command line at your terminal, the shell will prompt with the value of the shell variable PS2 if an end parenthesis is expected.

Braces ({ and }) can also group commands together. Both the left and the right brace are recognized *only* if they appear as the first (unquoted) word of a command. The opening brace may be followed by a newline (in which case the shell prompts for more input). Unlike parentheses, no subshell is created for braces: the enclosed commands are simply read by the shell. The braces are convenient when you wish to use the (sequential) output of several commands as input to one command.

The exit status of a set of commands grouped by either parentheses or braces is the exit status of the last enclosed executed command.

7.9.8 Input/Output Redirection and Control Commands

The shell normally does *not* fork and create a new shell when it recognizes the control commands (other than parentheses) described in the previous section. However, each command in a pipeline is run as a separate process in order to direct input to or output from each command. Also, when redirection of input or output is specified explicitly to a control command, a separate process is spawned to execute that command. Thus, when **if**, **while**, **until**, **case**, and **for** are used in a pipeline consisting of more than one command, the shell forks and a subshell runs the control command. This has two implications:

1. Any changes made to variables within the control command are not effective once that control command finishes (this is similar to the effect of using parentheses to group commands).
2. Control commands run slightly slower when redirected because of the additional overhead of creating a shell for the control command.

7.9.9 Transfer to Another File and Back: The Dot (.) Command

A command line of the form

```
. proc
```

causes the shell to read commands from *proc* without spawning a new process. Changes made to variables in *proc* are in effect after the dot command finishes. This is a good way to gather a number of shell variable initializations into one file. A common use of this command is to reinitialize the top level shell by reading the *.profile* file with

```
. profile
```

7.9.10 Interrupt Handling: trap

Shell procedures can use the **trap** command to disable a signal (cause it to be ignored), or redefine its action. The form of the **trap** command is

```
trap arg signal-list
```

arg is a string to be interpreted as a command list and *signal-list* consists of one or more signal numbers as described in *signal(S)* in the *XENIX Reference Manual*. The most important of these signals are

Number	Signal
--------	--------

- 00 KILL (CTRL-U)
- 01 HANGUP
- 02 INTERRUPT character
- 03 QUIT
- 09 KILL (cannot be caught or ignored)
- 11 segmentation violation (cannot be caught or ignored)
- 15 software termination signal

The commands in *arg* are scanned at least once, when the shell first encounters the **trap** command. Because of this, it is usually wise to surround these command with single rather than double quotation marks. The former inhibit immediate command and variable substitution. This becomes important, for instance, when you want to remove temporary files when the names of those files have not yet been determined when the **trap** command is first read by the shell. The following procedure will print the name of the current directory in the file *errdirect* when it is interrupted, thus giving the user information as to how much of the job was done:

```
trap `echo `pwd` >errdirect` 2 3 15
for i in /bin /usr/bin /usr/gas/bin
do
    cd $i
    # commands to be executed in directory $i here
done
```

Beware that the same procedure with double rather than single quotation marks does something different. The following prints the name of the directory from which the procedure was first executed:

```
(trap "echo `pwd` >errdirect" 2 3 15)
```

A signal 11 can never be trapped, because the shell itself needs to catch it to deal with memory allocation. Zero is interpreted by the **trap** command as a signal generated by exiting from a shell. This occurs either with an **exit** command, or by “falling through” to the end of a procedure. If *arg* is not specified, then the action taken upon receipt of any of the signals in the signal list is reset to the default system action. If *arg* is an explicit null string (`` or '') or colon (:), then the signals in the signal list are ignored by the shell.

The **trap** command is most frequently used to make sure that temporary files are removed upon termination of a procedure. The preceding example would

be written more typically as follows:

```
temp=$HOME/temp/$$
trap 'rm $temp; trap 0; exit' 0 1 2 3 15
ls > $temp
# commands that use $temp here
```

In this example, whenever signal 1 (hangup), 2 (interrupt), 3 (quit), or 15 (terminate) is received by the shell procedure, or whenever the shell procedure is about to exit, the commands enclosed between the single quotation marks are executed. The **exit** command must be included, or else the shell continues reading commands where it left off when the signal was received. The “trap 0” in the above procedure turns off the original traps 1, 2, 3, and 15 on exits from the shell, so that the **exit** command does not reactivate the execution of the trap commands.

Sometimes the shell continues reading commands after executing trap commands. The following procedure takes each directory in the current directory, changes to that directory, prompts with its name, and executes commands typed at the terminal until an end-of-file (CTRL-D) or an interrupt is received. An end-of-file causes the **read** command to return a nonzero exit status, and thus the **while** loop terminates and the next directory cycle is initiated. An interrupt is ignored while executing the requested commands, but causes termination of the procedure when it is waiting for input:

```
d=`pwd`
for i in *
do    if test -d $d/$i
      then cd $d/$i
        while echo "$i:"
              trap exit 2
              read x
              do trap : 2
                # ignore interrupts
                eval $x
              done
        fi
      done
done
```

Several traps may be in effect at the same time: if multiple signals are received simultaneously, they are serviced in numerically ascending order. To determine which traps are currently set, type

trap

It is important to understand the way in which the shell implements the **trap**

command. When a signal (other than 11) is received by the shell, it is passed on to whatever child processes are currently executing. When these (synchronous) processes terminate, normally or abnormally, the shell polls any traps that happen to be set and executes the appropriate **trap** commands. This process is straightforward, except in the case of traps set at the command (outermost, or login) level. In this case, it is possible that no child process is running, so before the shell polls the traps, it waits for the termination of the first process spawned *after* the signal was received.

When a signal is redefined in a shell script, this does not redefine the signal for programs invoked by that script; the signal is merely passed along. A disabled signal is not passed.

For internal commands, the shell normally polls traps on completion of the command. An exception to this rule is made for the **read** command, for which traps are serviced immediately, so that **read** can be interrupted while waiting for input.

7.10 Special Shell Commands

Several special commands are *internal* to the shell. The shell does not fork to execute these commands, so no additional processes are spawned. These commands should be used whenever possible, because they are, in general, faster and more efficient than other XENIX commands. The trade-off for this efficiency is that redirection of input and output is not allowed for most of these special commands.

Several of the special commands have already been described because they affect the flow of control. They are **dot**(.), **break**, **continue**, **exit**, and **trap**. The **set** command is also a special command. The remaining special commands are

- : Does nothing (the **null** command). Can be used to insert comments in shell procedures. Its exit status is zero (true). Its utility as a comment character has largely been supplanted by the number sign (#) which can insert comments to the end-of-line. Beware: any arguments to the null command are parsed for syntactic correctness; when in doubt, quote such arguments. Parameter substitution takes place, just as in other commands.
- cd arg** Makes *arg* the current directory. If *arg* is not a valid directory, or the user is not authorized to access it, a nonzero exit status is returned. Specifying **cd** with no *arg* is equivalent to typing "**cd \$HOME**" which takes you to your

home directory.

exec *arg* ...

Executes the command (if *arg* is a command) without forking and returning to the current shell. This is effectively a “goto” and no new process is created. Input and output redirection arguments are allowed on the command line. If *only* input and output redirection arguments appear, then the input and output of the shell itself are modified accordingly.

newgrp *arg* ...

Executes the **newgrp** command, replacing the shell. **Newgrp** in turn creates a new shell. Beware: only environment variables will be known in the shell created by the **newgrp** command. Any exported variables will no longer be marked as such.

read *var* ...

Reads one line (up to a newline) from the standard input and assigns the first word to the first variable, the second word to the second variable, and so on. All words left over are assigned to the *last* variable. The exit status of **read** is zero unless an end-of-file is read.

readonly *var* ...

Makes the specified variables **readonly** so that no subsequent assignments may be made to them. If no arguments are given, a list of all **readonly** and of all exported variables is given.

times

Prints the accumulated user and system times for processes run from the current shell.

umask *nnn*

Sets the user file creation mask to *nnn*. If *nnn* is omitted, the current value of the mask is printed. This bit-mask is used to set the default permissions when creating files. For example, an octal umask of 137 corresponds to the following bit-mask and permission settings for a newly created file:

User	User	Group	Other
Octal	1	3	7
bit-mask	001	011	111
permissions	rw-	r--	---

See *umask(C)* in the *XENIX Reference Manual* for information on the value of *nnn*.

`wait`

Makes the shell wait for all currently active child processes to terminate. The exit status of `wait` is always zero.

7.11 Creation and Organization of Shell Procedures

A shell procedure can be created in two simple steps. The first is building an ordinary text file. The second is changing the *mode* of the file to make it *executable*, thus permitting it to be invoked by

```
proc args
```

rather than

```
sh proc args
```

The second step may be omitted if a procedure is to be used once or twice and then discarded, but is recommended for frequently-used ones. To set up a simple procedure, first create a file named *mailall* with the following contents:

```
LETTER=$1
shift
for i in $*
do mail $i <$LETTER
done
```

Next type:

```
chmod +x mailall
```

The new command can then be invoked from within the current directory by typing

```
mailall letter joe bob
```

letter is the name of the file containing the message you want to send, and *joe* and *bob* are people you want to send the message to. Note that shell procedures must always be at least readable, so that the shell itself can read commands from the file.

If *mailall* were thus created in a directory whose name appears in the user's

PATH variable, the user could change working directories and still invoke the *mailall* command.

Shell procedures may be created dynamically. A procedure may generate a file of commands, and then invoke another instance of the shell to execute and remove the file. Or the **dot** command (.) can make the current shell read commands from the new file, which allows use of existing shell variables and avoids the spawning of an additional process for another shell.

Many users prefer writing shell procedures to writing C programs. This is true for several reasons:

1. A shell procedure is easy to create and maintain because it is only a file of ordinary text.
2. A shell procedure has no corresponding object program that must be generated and maintained.
3. A shell procedure is easy to create quickly, use a few times, and then remove.
4. Because shell procedures are usually short in length, written in a high-level programming language, and kept only in their source-language form, they are generally easy to find, understand, and modify.

By convention, directories that contain only commands and shell procedures are named *bin*. This name is derived from the word “binary,” and is used because compiled and executable programs are often called “binaries” to distinguish them from program source files. Most groups of users sharing common interests have one or more *bin* directories set up to hold common procedures. Some users have their PATH variable list several such directories, although an excess of them may cause a decrease in system efficiency.

7.12 More About Execution Flags

Several execution flags available in the shell can be useful in shell procedures:

- e This flag causes the shell to exit immediately if any command that it executes exits with a nonzero exit status. This flag is useful for shell procedures composed of simple command lines; it is not intended for use in conjunction with other conditional constructs.
- u This flag causes unset variables to be considered errors when substituting variable values. This flag can be used to effect a global check on variables, rather than using conditional substitution to check each variable.

- t** This flag causes the shell to exit after reading and executing the commands on the remainder of the current input line. This flag is typically used by C programs which call the shell to execute a single command.
- n** This is a “don’t execute” flag. On occasion, you may want to check a procedure for syntax errors without executing the commands in the procedure. Using “set -nv” at the beginning of a file will accomplish this.
- k** This flag causes all arguments of the form *variable=value* to be treated as keyword parameters. When this flag is *not* set, only such arguments that appear before the command name are treated as keyword parameters.

7.13 Supporting Commands and Features

Shell procedures can make use of any XENIX command. The commands described in this section are used frequently in shell procedures, or have been explicitly designed for such use.

7.13.1 Conditional Evaluation: test

The **test** command evaluates the expression specified by its arguments and returns a zero exit status if the expression is true. If the expression is false, **test** returns a nonzero (false) exit status. **Test** also returns a nonzero exit status if it has no arguments. Often it is convenient to use the **test** command as the first command in the command list following an **if** or a **while**. Shell variables used in **test** expressions should be enclosed in double quotation marks if there is any chance of their being null or not set.

The square brackets may be used as an alias to **test**, so that

[*expression*]

has the same effect as

test *expression*

Note that the spaces before and after the *expression* in brackets are essential.

The following is a partial list of the options that can be used to construct a conditional expression:

- r file** True if the named file exists and is readable by the user.
- w file** True if the named file exists and is writable by the

	user.
-x <i>file</i>	True if the named file exists and is executable by the user.
-s <i>file</i>	True if the named file exists and has a size greater than zero.
-d <i>file</i>	True if the named file is a directory.
-f <i>file</i>	True if the named file is an ordinary file.
-z <i>s1</i>	True if the length of string <i>s1</i> is zero.
-n <i>s1</i>	True if the length of the string <i>s1</i> is nonzero.
-t <i>fildev</i>	True if the open file whose file descriptor number is <i>fildev</i> is associated with a terminal device. If <i>fildev</i> is not specified, file descriptor 1 is used by default.
<i>s1</i> = <i>s2</i>	True if strings <i>s1</i> and <i>s2</i> are identical.
<i>s1</i> != <i>s2</i>	True if strings <i>s1</i> and <i>s2</i> are <i>not</i> identical.
<i>s1</i>	True if <i>s1</i> is <i>not</i> the null string.
<i>n1</i> -eq <i>n2</i>	True if the integers <i>n1</i> and <i>n2</i> are algebraically equal; other algebraic comparisons are indicated by -ne (not equal), -gt (greater than), -ge (greater than or equal to), -lt (less than), and -le (less than or equal to).

The options can be combined with the following operators:

!	Unary negation operator.
-a	Binary logical AND operator.
-o	Binary logical OR operator; it has lower precedence than the logical AND operator (-a).
(<i>expr</i>)	Parentheses for grouping; they must be escaped to remove their significance to the shell. In the absence of parentheses, evaluation proceeds from left to right.

Note that all options, operators, filenames, etc. are separate arguments to **test**.

7.13.2 Echoing Arguments

The **echo** command has the following syntax:

```
echo [options][args]
```

Echo copies its arguments to the standard output. Each is followed by a single space, except for the last argument, which is normally followed by a newline. **Echo** is often used to prompt the user for input, to issue diagnostics in shell procedures, or to add a few lines to an output stream in the middle of a pipeline. Another use is to verify the argument list generation process before issuing a command that does something drastic. The command

```
ls
```

is often replaced by

```
echo *
```

because the latter is faster and prints fewer lines of output.

The **-n** option to **echo** removes the newline from the end of the echoed line. Thus, the following two commands prompt for input and then allow typing on the same line as the prompt:

```
echo -n 'enter name:'
read name
```

The **echo** command also recognizes several escape sequences described in *echo(C)* in the *XENIX Reference Manual*.

7.13.3 Expression Evaluation: **expr**

The **expr** command provides arithmetic and logical operations on integers and some pattern-matching facilities on its arguments. It evaluates a single expression and writes the result on the standard output. **expr** can be used inside grave accents to set a variable. Some typical examples follow:

```
#      increment $A
A=`expr $A + 1`
#      put third through last characters of
#      $1 into substring
substring=`expr "$1" : '..\(.*\) ^`
#      obtain length of $1
c=`expr "$1" : '.*'^`
```

The most common uses of **expr** are in counting iterations of a loop and in us-

ing its pattern-matching capability to pick apart strings.

7.13.4 True and False

The **true** and **false** commands perform the functions of exiting with zero and nonzero exit status, respectively. The **true** and **false** commands are often used to implement unconditional loops. For example, you might type

```
while true
do echo forever
done
```

This will echo “forever” on the screen until an INTERRUPT is typed.

7.13.5 In-Line Input Documents

Upon seeing a command line of the form

```
command<<eofstring
```

where *eofstring* is any arbitrary string, the shell will take the subsequent lines as the standard input of *command* until a line is read consisting only of *eofstring*. (By appending a minus [-] to the input redirection symbol [<<], leading tabs are deleted from each line of the input document before the shell passes the line to *command*.)

The shell creates a temporary file containing the input document and performs variable and command substitution on its contents before passing it to the command.

Pattern matching on filenames is performed on the arguments of command lines in command substitutions. In order to prohibit all substitutions, you may quote any character of *eofstring*:

```
command<<\\eofstring
```

The in-line input document feature is especially useful for small amounts of input data, where it is more convenient to place the data in the shell procedure than to keep it in a separate file. For instance, you could type

```
cat <<-xx
    This message will be printed on the
    terminal with leading tabs removed.
xx
```

This in-line input document feature is most useful in shell procedures. Note that in-line input documents may not appear within grave accents.

7.13.6 Input/Output Redirection Using File Descriptors

We mentioned above that a command occasionally directs output to some file associated with a file descriptor other than 1 or 2. In languages such as C, one can associate output with any file descriptor by using the *write(S)* system call (see the *XENIX Reference Manual*). The shell provides its own mechanism for creating an output file associated with a particular file descriptor. By typing

```
fd1>&fd2
```

where *fd1* and *fd2* are valid file descriptors, one can direct output that would normally be associated with file descriptor *fd1* to the file associated with *fd2*. The default value for *fd1* and *fd2* is 1. If, at run time, no file is associated with *fd2*, then the redirection is void.

The most common use of this mechanism is that of directing standard error output to the same file as standard output. This is accomplished by typing

```
command 2>&1
```

If you wanted to redirect both standard output and standard error output to the same file, you would type

```
command 1>file 2>&1
```

The order here is significant: first, file descriptor 1 is associated with *file*; then file descriptor 2 is associated with the same file as is currently associated with file descriptor 1. If the order of the redirections were reversed, standard error output would go to the terminal, and standard output would go to *file*, because at the time of the error output redirection, file descriptor 1 still would have been associated with the terminal.

This mechanism can also be generalized to redirect standard input. You could type

```
fda<&fdb
```

to cause both file descriptors *fda* and *fdb* to be associated with the same input file. If *fda* or *fdb* is not specified, file descriptor 0 is assumed. Such input redirection is useful for a command that uses two or more input sources.

7.13.7 Conditional Substitution

Normally, the shell replaces occurrences of *\$variable* by the string value assigned to *variable*, if any. However, a special notation allows conditional substitution, depending on whether the variable is set or not null. By definition, a variable is set if it has ever been assigned a value. The value of a vari-

able can be the null string, which may be assigned to a variable in anyone of the following ways:

```
A=
bcd=""
efg=""
set "" ""
```

The first three examples assign null to each of the corresponding shell variables. The last example sets the first and second positional parameters to null. The following conditional expressions depend on whether a variable is set and not null. Note that the meaning of braces in these expressions differs from their meaning when used in grouping shell commands, and that here, *parameter* refers to either a digit or a variable name.

`${variable:-string}`

If *variable* is set and is nonnull, then substitute the value *\$variable* in place of this expression. Otherwise, replace the expression with *string*. Note that the value of *variable* is *not* changed by the evaluation of this expression.

`${variable:=string}`

If *variable* is set and is nonnull, then substitute the value *\$variable* in place of this expression. Otherwise, set *variable* to *string* and then substitute the value *\$variable* in place of this expression. Positional parameters may not be assigned values in this fashion.

`${variable:?string}`

If *variable* is set and is nonnull, then substitute the value of *variable* for the expression. Otherwise, print a message of the form

variable: string

and exit from the current shell. (If the shell is the login shell, it is not exited.) If *string* is omitted in this form, then the message

variable: parameter null or not set

is printed instead.

`${variable:+string}`

If *variable* is set and is nonnull, then substitute *string* for this expression. Otherwise, substitute the null string. Note that the value of *variable* is not altered by the

evaluation of this expression.

These expressions may also be used without the colon. In this variation, the shell does not check whether the variable is null or not; it only checks whether the variable has ever been set.

The following examples illustrate the use of this facility:

1. This example performs an explicit assignment to the PATH variable:

```
PATH=${PATH:-~/bin:/usr/bin}
```

This says, if PATH has ever been set and is not null, then it keeps its current value; otherwise, set it to the string “~/bin:/usr/bin”.

2. This example automatically assigns the HOME variable a value:

```
cd ${HOME:=~/usr/gas}
```

If HOME is set, and is not null, then change directory to it. Otherwise set HOME to the given value and change directory to it.

7.13.8 Invocation Flags

Four flags can be specified on the shell command line. These flags may *not* be turned on with the `set` command:

- i If this flag is specified, or if the shell's input and output are both attached to a terminal, the shell is *interactive*. In such a shell, INTERRUPT (signal 2) is caught and ignored, and TERMINATE (signal 15) and QUIT (signal 3) are ignored.
- s If this flag is specified or if no input/output redirection arguments are given, the shell reads commands from standard input. Shell output is written to file descriptor 2. The shell you get upon logging into the system has the –s flag turned on.
- c When this flag is turned on, the shell reads commands from the first string following the flag. Remaining arguments are ignored. Use double quotation marks to enclose a multiword string in order to allow for variable substitution.

7.14 Effective and Efficient Shell Programming

This section outlines strategies for writing efficient shell procedures, ones that do not waste resources in accomplishing their purposes. The primary reason for choosing a shell procedure to perform a specific function is to achieve a desired result at a minimum human cost. Emphasis should always be placed

on simplicity, clarity, and readability, but efficiency can also be gained through awareness of a few design strategies. In many cases, an effective redesign of an existing procedure improves its efficiency by reducing its size, and often increases its comprehensibility. In any case, you needn't plan to optimize shell procedures unless they are intolerably slow or are known to consume an inordinate amount of system resources.

Shell procedures should undergo the same iteration cycle as other programs: write code, measure it, and optimize only the *few* important parts. The user should become familiar with the **time** command, which can be used to measure both entire procedures and parts thereof. Its use is strongly recommended; human intuition is notoriously unreliable when used to estimate timings of programs, even when the style of programming is a familiar one. Each timing test should be run several times, because the results are easily disturbed by variations in system load.

7.14.1 Number of Processes Generated

When large numbers of short commands are executed, the actual execution time of the commands may well be dominated by the overhead of creating processes. The procedures that incur significant amounts of such overhead are those that perform much looping, and those that generate command sequences to be interpreted by another shell.

If you are worried about efficiency, it is important to know which commands are currently built into the shell, and which are not. Here is an alphabetical list of built-in commands:

break	exec	newgrp	test	wait
case	exit	read	times	while
cd	export	readonly	trap	.
continue	for	set	umask	:
eval	if	shift	until	{ }

Parentheses, (), are built into the shell, but commands enclosed within them are executed as a child process, i.e., the shell does a **fork**, but no **exec**. Any command not in the above list requires both **fork** and **exec**.

You should always have at least a vague idea of the number of processes generated by a shell procedure. In the bulk of observed procedures, the number of processes created (not necessarily simultaneously) can be described by

$$\text{processes} = (k * n) + c$$

where k and c are constants, and n may be the number of procedure arguments, the number of lines in some input file, the number of entries in some directory, or some other obvious quantity. Efficiency improvements are most commonly gained by reducing the value of k , sometimes to zero.

Any procedure whose complexity measure includes n^2 terms or higher powers of n is likely to be intolerably expensive.

Some types of procedures should *not* be written using the shell. For example, if one or more processes are generated for each character in a file, it is a good indication that the procedure should be rewritten in C. Shell procedures should not be used to scan or build files a character at a time.

7.14.2 Number of Data Bytes Accessed

It is worthwhile to consider any action that reduces the number of bytes read or written. This may be important for those procedures whose time is spent passing data around among a few processes, rather than in creating large numbers of short processes. Some filters shrink their output, others usually increase it. It always pays to put the *shrinkers* first when the order is irrelevant. For instance, the second of the following examples is likely to be faster since the input to `sort` will be much smaller:

```
sort file | grep pattern
grep pattern file | sort
```

7.14.3 Shortening Directory Searches

Directory searching can consume a great deal of time, especially in those applications that utilize deep directory structures and long pathnames. Judicious use of `cd`, the **change directory** command, can help shorten long pathnames and thus reduce the number of directory searches needed. As an exercise, try the following commands:

```
ls -l /usr/bin/* >/dev/null
cd /usr/bin; ls -l * >/dev/null
```

The second command will run faster because of the fewer directory searches.

7.14.4 Directory Search Order and the PATH Variable

The `PATH` variable is a convenient mechanism for allowing organization and sharing of procedures. However, it must be used in a sensible fashion, or the result may be a great increase in system overhead.

The process of finding a command involves reading every directory included in every pathname that precedes the needed pathname in the current `PATH` variable. As an example, consider the effect of invoking `nroff` (i.e., `/usr/bin/nroff`) when the value of `PATH` is `“.:bin:/usr/bin”`. The sequence of directories read is

```

.  
/  
/bin  
/  
/usr  
/usr/bin

```

This is a total of six directories. A long path list assigned to `PATH` can increase this number significantly.

The vast majority of command executions are of commands found in `/bin` and, to a somewhat lesser extent, in `/usr/bin`. Careless `PATH` setup may lead to a great deal of unnecessary searching. The following four examples are ordered from worst to best with respect to the efficiency of command searches:

```

:/usr/john/bin:/usr/localbin:/bin:/usr/bin  
:/bin:/usr/john/bin:/usr/localbin:/usr/bin  
:/bin:/usr/bin:/usr/john/bin:/usr/localbin  
/bin::/usr/bin:/usr/john/bin:/usr/localbin

```

The first one above should be avoided. The others are acceptable and the choice among them is dictated by the rate of change in the set of commands kept in `/bin` and `/usr/bin`.

A procedure that is expensive because it invokes many short-lived commands may often be speeded up by setting the `PATH` variable inside the procedure so that the fewest possible directories are searched in an optimum order.

7.14.5 Good Ways to Set Up Directories

It is wise to avoid directories that are larger than necessary. You should be aware of several special sizes. A directory that contains entries for up to 30 files (plus the required `.` and `..`) fits in a single disk block and can be searched very efficiently. One that has up to 286 entries is still a small directory; anything larger is usually a disaster when used as a working directory. It is especially important to keep login directories small, preferably one block at most. Note that, as a rule, directories never shrink. This is very important to understand, because if your directory ever exceeds either the 30 or 286 thresholds, searches will be inefficient; furthermore, even if you delete files so that the number of files is less than either threshold, the system will still continue to treat the directory inefficiently.

7.15 Shell Procedure Examples

The power of the XENIX shell command language is most readily seen by examining how XENIX's many labor-saving utilities can be combined to perform powerful and useful commands with very little programming effort. This section gives examples of procedures that do just that. By studying these examples, you will gain insight into the techniques and shortcuts that can be used in programming shell procedures (also called "scripts"). Note the use of the number sign (#) to introduce comments into shell procedures.

Carry out the following steps for each procedure:

1. Place the procedure in a file with the indicated name.
2. Give the file execute permission with the **chmod** command.
3. Move the file to a directory in which commands are kept, such as your own *bin* directory.
4. Make sure that the path of the *bin* directory is specified in the **PATH** variable found in *.profile*.
5. Execute the named command.

BINUNIQ

```
ls /bin /usr/bin | sort | uniq -d
```

This procedure determines which files are in both */bin* and */usr/bin*. It is done because files in */bin* will override those in */usr/bin* during most searches, and duplicates need to be weeded out. If the */usr/bin* file is obsolete, then space is being wasted; if the */bin* file is outdated by a corresponding entry in */usr/bin*, then the wrong version is being run and, again, space is being wasted. This is also a good demonstration of "sort | uniq" to find matches and duplications.

COPYPAIRS


```

#      Usage: cypairs file1 file2 ...
#      Copies file1 to file2, file3 to file4, ...
while test "$2" != ""
do
    cp $1 $2
    shift; shift
done
if test "$1" != ""
then echo "$0: odd number of arguments"
fi

```

This procedure illustrates the use of a **while** loop to process a list of positional parameters that are somehow related to one another. Here a **while** loop is much better than a **for** loop, because you can adjust the positional parameters with the **shift** command to handle related arguments.

COPYTO

```

#      Usage: copyto dir file ...
#      Copies argument files to "dir",
#      making sure that at least
#      two arguments exist, that "dir" is a directory,
#      and that each additional argument
#      is a readable file.
if test $# -lt 2
then    echo "$0: usage: copyto directory file ..."
elif test ! -d $1
then    echo "$0: $1 is not a directory";
else
    dir=$1; shift
    for eachfile
    do    cp $eachfile $dir
done
fi

```

This procedure uses an **if** command with several parts to screen out improper usage. The **for** loop at the end of the procedure loops over all of the arguments

to **copyto** but the first; the original **\$1** is shifted off.

DISTINCT1

```
#      Usage: distinct1
#      Reads standard input and reports list of
#      alphanumeric strings that differ only in case,
#      giving lowercase form of each.
tr -cs '[A-Z][a-z][0-9]' '\[012*255]' | sort -u | \
tr '[A-Z]' '[a-z]' | sort | uniq -d
```

This procedure demonstrates the kind of process that is created by the left-to-right construction of a long pipeline. Note the use of the backslash at the end of the first line as the line continuation character. It may not be immediately obvious how this command works. You may wish to consult *tr(C)*, *sort(C)*, and *uniq(C)* in the *XENIX Reference Manual* if you are completely unfamiliar with these commands. The **tr** command translates all characters except letters and digits into newline characters, and then squeezes out repeated newline characters. This leaves each string (in this case, any contiguous sequence of letters and digits) on a separate line. The **sort** command sorts the lines and emits only one line from any sequence of one or more repeated lines. The next **tr** converts everything to lowercase, so that identifiers differing only in case become identical. The output is sorted again to bring such duplicates together. The “**uniq -d**” prints (once) only those lines that occur more than once, yielding the desired list.

The process of building such a pipeline relies on the fact that pipes and files can usually be interchanged. In the following example, the first line is equivalent to the last two lines, assuming that sufficient disk space is available:

```
cmd1 | cmd2 | cmd3
```

```
cmd1 > temp1; < temp1 cmd2 > temp2; < temp2 cmd3
rm temp[123]
```

Starting with a file of test data on the standard input and working from left to right, each command is executed taking its input from the previous file and putting its output in the next file. The final output is then examined to make sure that it contains the expected result. The goal is to create a series of transformations that will convert the input to the desired output.

Although pipelines can give a concise notation for complex processes, you should exercise some restraint, since such practice often yields incomprehensible code.

DRAFT This procedure illustrates taking input from the user and printing it back. It is a simple example of a procedure that is useful in many situations. It is a good example of how to use the shell to create a simple program.

```
#      Usage: draft file(s)
#      Print manual pages for Diablo printer.
for i in $*
do nroff -man $i | lpr
done
```

Users often write this kind of procedure for convenience in dealing with commands that require the use of distinct flags that cannot be given default values that are reasonable for all (or even most) users.

EDFIND

```
#      Usage: edfind file arg
#      Finds the last occurrence in "file" of a line
#      whose beginning matches "arg", then prints
#      the argument requested, followed by
#      3 lines (the one before, the line itself,
#      and the one after)
ed - $1 <<-EOF
    ?^$2?
    -,+p
    q
EOF
```

This illustrates the practice of using `ed` in-line input documents into which the shell can substitute the values of variables.

EDLAST

```
#      Usage: edlast file
#      Prints the last line of file,
#      then deletes that line.
ed - $1 <<-\!
    $p
    $d
    w
    q
echo done
```

This procedure illustrates taking input from within the file itself up to the exclamation point (!). Variable substitution is prohibited within the input text because of the backslash.

FSPLIT

```
#      Usage: fsplit file1 file2
#      Reads standard input and divides it into 3 parts
#      by appending any line containing at least one letter
#      to file1, appending any line containing digits but
#      no letters to file2, and by throwing the rest away.
count=0 gone=0
while read next
do
    count="`expr $count + 1 `"
    case "$next" in
        *[A-Za-z]*)
            echo "$next" >> $1 ;;
        *[0-9]*)
            echo "$next" >> $2 ;;
        *)
            gone="`expr $gone + 1 `"
    esac
done
echo "$count lines read, $gone thrown away"
```

Each iteration of the loop reads a line from the input and analyzes it. The loop terminates only when `read` encounters an end-of-file. Note the use of the `expr` command.

Don't use the shell to read a line at a time unless you must—it can be an extremely slow process.

LISTFIELDS

```
#
#      usage: listfields pattern [files]
#
grep $* | tr ":" "\012"
```

This procedure lists lines from files or the keyboard containing any desired entry that is given to it as the first argument. It places any field that begins with a colon on a newline. Thus, if the following line is in a file called "names,"

joe newman: 13509 NE 78th St: Redmond, Wa 98062

The command *listfields joe names* produces this:

```
joe newman
13509 NE 78th St
Redmond, Wa 98062
```

Note the use of the **tr** command to transpose colons to linefeeds.

MKFILES

```
#      Usage: mkfiles pref [quantity]
#      Makes "quantity" files, named pref1, pref2, ...
#      Default is 5 as determined on following line.
quantity=${2-5}
i=1
while test "$i" -le "$quantity"
do
    > $1$i
    i=$((expr $i + 1))
done
```

The *mkfiles* procedure uses output redirection to create zero-length files. The **expr** command is used for counting iterations of the **while** loop.

NULL

```
#      Usage: null files
#      Create each of the named files as an empty file.
for eachfile
do
    >$eachfile
done
```

This procedure uses the fact that output redirection creates the (empty) output

file if a file does not already exist.

PHONE

```
#      Usage: phone initials...
#      Prints the phone numbers of the
#      people with the given initials.
echo `init      ext      home`
grep "^$1" <<-END
      jfk      1234      999-2345
      lbj      2234      583-2245
      hst      3342      988-1010
      jqa      4567      555-1234
END
```

This procedure is an example of using an in-line input script to maintain a small data base.

TEXTFILE

```
if test "$1" = "-s"
then
#      Return condition code
shift
if test -z "$0 $*" # check return value
then
exit 1
else
exit 0
fi
fi
if test $# -lt 1
then
echo "$0: Usage: $0 [-s] file ..." 1>&2
exit 0
fi

file $* | fgrep `text` | sed 's/:      .*/'
```

To determine which files in a directory contain only textual information, *textfile* filters argument lists to other commands. For example, the following command line will print all the text files in the current directory:

```
pr `textfile *` | lpr
```

This procedure also uses an **-s** flag which silently tests whether any of the files in the argument list is a text file.

WRITEMAIL

```
#      Usage: writemail message user
#      If user is logged in,
#      writes message to terminal;
#      otherwise, mails it to user.
echo "$1" | { write "$2" || mail "$2"; }
```

This procedure illustrates the use of command grouping. The message specified by \$1 is piped to both the **write** command and, if **write** fails, to the **mail** command.

7.16 Shell Grammar

Feature	Syntax
<i>item:</i>	<i>word</i> <i>input-output</i> <i>name = value</i>
<i>simple-command:</i>	<i>item</i> <i>simple-command item</i>
<i>command:</i>	<i>simple-command</i> (<i>command-list</i>) { <i>command-list</i> } for <i>name</i> do <i>command-list</i> done for <i>name</i> in <i>word</i> do <i>command-list</i> done while <i>command-list</i> do <i>command-list</i> done until <i>command-list</i> do <i>command-list</i> done case <i>word</i> in <i>case-part</i> esac if <i>command-list</i> then <i>command-list</i> else-part fi
<i>pipeline:</i>	<i>command</i> <i>pipeline</i> <i>command</i>
<i>andor:</i>	<i>pipeline</i> <i>andor</i> && <i>pipeline</i> <i>andor</i> <i>pipeline</i>
<i>command-list:</i>	<i>andor</i> <i>command-list</i> ; <i>command-list</i> & <i>command-list</i> ; <i>andor</i> <i>command-list</i> & <i>andor</i>
<i>input-output:</i>	> <i>file</i> < <i>file</i> << <i>word</i> >> <i>word</i>
<i>file:</i>	<i>word</i> & <i>digit</i> & -
<i>case-part:</i>	<i>pattern</i>) <i>command-list</i> ;;
<i>pattern:</i>	<i>word</i> <i>pattern</i> <i>word</i>

else-part:

elif *command-list* **then** *command-list* *else-part*
else *command-list*
empty

empty:

word:

a sequence of nonblank characters

name:

a sequence of letters, digits, or underscores
starting with a letter

digit:

0 1 2 3 4 5 6 7 8 9

Metacharacters and Reserved Words

Syntax

	Pipe symbol
&&	And-if symbol
	Or-if symbol
;	Command separator
::	Case delimiter
&	Background commands
()	Command grouping
<	Input redirection
<<	Input document from here
>	Output creation
>>	Output append
#	Comment to end of line

Patterns

*	Match any character(s) including none
?	Match any single character
[...]	Match any of the enclosed characters

Substitution

\${...}	Substitute shell variable
`...`	Substitute command output

Quoting

\	Quote next character as literal with no special meaning
'...'	Quote enclosed characters excepting the single, open quotation marks (')
"..."	Quote enclosed characters excepting: \$`\"

Reserved Words

case	for
do	if
done	in
elif	then
else	until
esac	while
fi	{ }

